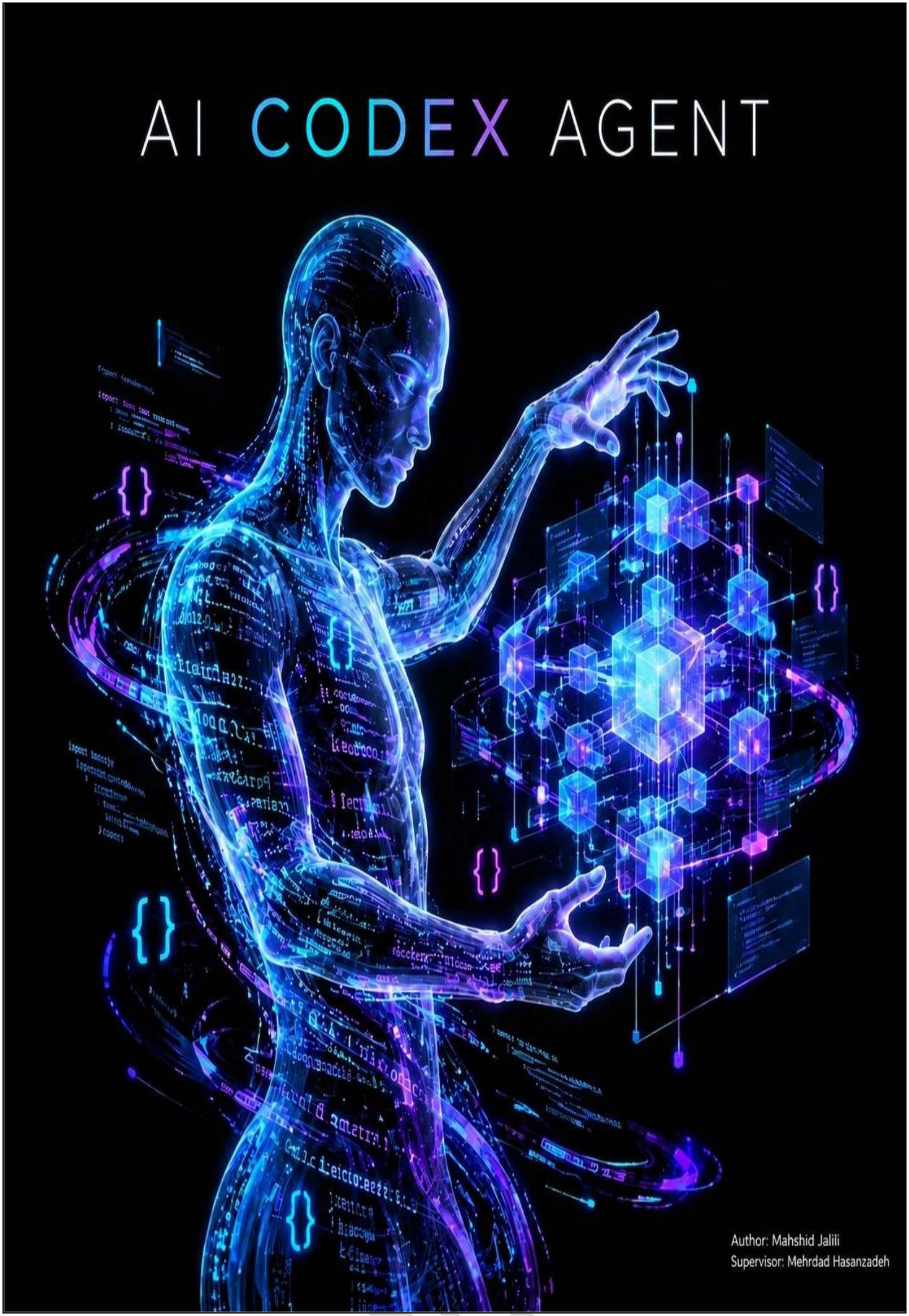
AI CODEX AGENT



Author: Mahshid Jalili
Supervisor: Mehrdad Hasanzadeh

یادداشت دربارهٔ این کتابچه

این کتابچه با هدف آموزش عملی و ایمن کار با Codex در پروژه‌های نرم‌افزاری تهیه شده است. محتوای آن بر پایهٔ الگوهای رایج کار با عامل‌های هوشمند کدنویسی، مدیریت پروژه، Git، محیط‌های اجرای محلی و ابری و قابلیت‌های پیشرفته‌ای مانند Skill، Subagent، Plugin و MCP تنظیم شده است.

برخی قابلیت‌ها، نام دستورها، مدل‌ها، گزینه‌های دسترسی، سهمیه‌ها و جزئیات محیط اجرای Codex ممکن است با انتشار نسخه‌های جدید تغییر کنند. بنابراین، دستورها یا قابلیت‌هایی که در این کتابچه به نسخهٔ خاصی وابسته هستند باید با مستندات رسمی و نسخهٔ نصب‌شدهٔ کاربر مقایسه شوند.

این کتابچه جایگزین مستندات رسمی، بررسی انسانی کد یا سیاست‌های امنیتی سازمان نیست. کاربر مسئول بررسی تغییرات، اجرای تست‌ها، حفاظت از اطلاعات حساس و تأیید نهایی عملیات Git است.

bitwithit

فهرست مطالب

پیشگفتار

فصل اول: آشنایی با Codex

۱.۱. Codex چیست؟

۱.۲. Codex چه مسئله‌ای را حل می‌کند؟

۱.۳. Codex برای چه کسانی مناسب است؟

۱.۴. Codex چه کارهایی می‌تواند انجام دهد؟

۱.۵. محدودیت‌ها و مسئولیت کاربر

۱.۶. اصول طلایی کار با Codex

فصل دوم: محیط‌های کاری Codex

۲.۱. اپلیکیشن دسکتاپ

۲.۲. محیط CLI

۲.۳. محیط Cloud

۲.۴. Worktree

۲.۵. مقایسهٔ محیط‌های اجرا

۲.۶. مرورگر، موبایل و قابلیت صوتی

فصل سوم: راه‌اندازی نخستین پروژه

۳.۱. پیش‌نیازها

۳.۲. انتخاب ریشهٔ ریپو

۳.۳. بررسی وضعیت اولیهٔ Git

۳.۴. ایجاد شاخهٔ کاری

۳.۵. بررسی وضعیت Codex

۳.۶. ایجاد و بررسی AGENTS.md

فصل چهارم: چرخهٔ استاندارد انجام کار

bitwit

۴.۱. تعریف هدف

۴.۲. برنامه‌ریزی با /plan

۴.۳. اجرای هدف با /goal

۴.۴. بررسی تغییرات

۴.۵. اجرای تست

۴.۶. بازبینی با /review

۴.۷. commit دستی

فصل پنجم: راهنمای دستورهای اسلش و CLI

۵.۱. تفاوت دستور اسلش و دستور CLI

۵.۲. دستورهای هدف و برنامه‌ریزی

۵.۳. دستورهای وضعیت و بازبینی

۵.۴. دستورهای مدل و استدلال

۵.۵. دستورهای محیط اجرا

۵.۶. دستورهای مدیریت نشست

۵.۷. دستورهای ابزار و حافظه

۵.۸. دستورهای CLI

فصل ششم: کنترل دسترسی و امنیت

۶.۱. Sandbox چیست؟

۶.۲. Approval Policy چیست؟

۶.۳. سطوح دسترسی متداول

۶.۴. Computer Use

۶.۵. مدیریت فایل‌های حساس

فصل هفتم: Git و سه بررسی ضروری پیش از commit

۷.۱. بررسی اول: آیا فایل درست تغییر کرده است؟

۷.۲. بررسی دوم: آیا فایل حساس سالم است؟

۷.۳. بررسی سوم: آیا اندازهٔ تغییر منطقی است؟

۷.۴. بازگردانی تغییرات ناخواسته

۷.۵. کارهایی که نباید بدون بررسی به Codex سپرد

bitwitit

فصل هشتم: AGENTS.md و قوانین پایدار پروژه

۸.۱. نقش AGENTS.md

۸.۲. ترتیب خواندن فایل‌های راهنما

۸.۳. محتوای پیشنهادی

۸.۴. نمونهٔ کامل AGENTS.md

۸.۵. اشتباهات رایج

فصل نهم: قابلیت‌های پیشرفته

۹.۱. Skill

۹.۲. Subagent

۹.۳. Plugin

۹.۴. MCP

۹.۵. ترتیب پیشنهادی یادگیری قابلیت‌های پیشرفته

فصل دهم: انتخاب مدل و سطح Reasoning

۱۰.۱. انتخاب مدل

۱۰.۲. مفهوم Reasoning Effort

فصل یازدهم: الگوهای آماده برای استفاده

۱۱.۱. خطای درک مسعله

۱۱.۲. خطای محیط

۱۱.۳. خطای وابستگی

۱۱.۴. خطای کد

۱۱.۵. خطای محیط اجرایی و استقرار

۱۱.۶. خطاهای مربوط به مسیر و فایل

۱۱.۷.خطاهای مربوط به زمینه گفت وگو

پیشگفتار

دربارهٔ این کتابچه

bitwithit

ابزارهای هوش مصنوعی مولد در سال‌های اخیر از پاسخ‌گویی متنی ساده فراتر رفته‌اند و وارد چرخهٔ واقعی توسعهٔ نرم‌افزار شده‌اند. امروزه یک عامل هوشمند می‌تواند ساختار پروژه را بررسی کند، فایل‌های مرتبط را پیدا کند، برنامهٔ تغییرات پیشنهاد دهد، کد بنویسد، تست اجرا کند، خطاها را تحلیل کند و نتیجهٔ کار را برای بازبینی انسان آماده کند.

Codex یکی از ابزارهایی است که برای همین نوع تعامل طراحی شده است. با این حال، استفادهٔ مؤثر از Codex فقط به نوشتن یک درخواست در چت محدود نمی‌شود. برای گرفتن نتیجهٔ قابل‌اعتماد، کاربر باید بداند پروژه را چگونه معرفی کند، هدف را چگونه محدود کند، مجوزهای مناسب را چگونه انتخاب کند، تغییرات را چگونه بررسی کند و در چه نقطه‌ای نتیجهٔ کار را بپذیرد یا رد کند.

این کتابچه با همین هدف نوشته شده است: تبدیل کار با Codex از یک گفت‌وگوی پراکنده به یک فرایند منظم، قابل‌کنترل و قابل‌تکرار.

هدف کتابچه

هدف اصلی این کتابچه آموزش یک روش عملی برای کار با Codex است؛ روشی که در آن:

- هدف کار از ابتدا مشخص باشد.
- دامنهٔ تغییرات محدود باشد.
- فایل‌های مجاز و غیرمجاز تعریف شوند.
- اجرای تست بخشی از تعریف «تمام‌شدن کار» باشد.
- تغییرات Git قبل از commit بررسی شوند.
- عملیات حساس بدون تأیید کاربر انجام نشوند.
- قابلیت‌های پیشرفته فقط زمانی استفاده شوند که واقعاً ارزش ایجاد کنند.

در این کتابچه تلاش شده است توضیحات مفهومی در کنار دستورها، مثال‌ها، الگوهای آماده و چک‌لیست‌های عملی قرار بگیرند تا خواننده بتواند مطالب را مستقیماً در پروژه‌های شخصی یا تیمی خود به کار بگیرد.

این کتابچه برای چه کسانی است؟

این کتابچه برای گروه‌های زیر مناسب است:

- برنامه‌نویسان مبتدی که می‌خواهند کار با عامل‌های کدنویسی را به‌صورت اصولی آغاز کنند.
- توسعه‌دهندگان حرفه‌ای که به دنبال افزایش سرعت کارهای تکراری هستند.
- مهندسانی که با Python، JavaScript، TypeScript، Java، C، C++، Go یا زبان‌های دیگر کار می‌کنند.
- دانشجویان مهندسی نرم‌افزار، علوم کامپیوتر و مهندسی پزشکی.
- تیم‌هایی که قصد دارند استفاده از Codex را در فرایند توسعهٔ خود استاندارد کنند.
- مدیران فنی و رهبران تیم که به کنترل دسترسی، بازبینی و امنیت اهمیت می‌دهند.
- افرادی که می‌خواهند از محیط‌های Local، Worktree و Cloud به‌شکل هدفمند استفاده کنند.

برای استفاده از این کتابچه لازم نیست از ابتدا با همهٔ قابلیت‌های Codex آشنا باشید. مباحث از مفاهیم ساده شروع می‌شوند و به‌تدریج به قابلیت‌هایی مانند Skill، Subagent، Plugin و MCP می‌رسند.

بعد از مطالعهٔ این کتابچه چه خواهید آموخت؟

پس از مطالعهٔ کتابچه، باید بتوانید:

1. محیط مناسب کار با Codex را انتخاب کنید.
2. پروژه را با ساختار صحیح در اختیار Codex قرار دهید.
3. هدف کار را دقیق و محدود تعریف کنید.
4. برای کارهای چندمرحله‌ای برنامه‌ریزی کنید.
5. از دستورهای اسلش و CLI در جای مناسب استفاده کنید.
6. مجوزهای Sandbox و Approval Policy را درک کنید.
7. فایل AGENTS.md را برای قوانین پایدار پروژه تنظیم کنید.
8. تغییرات ایجادشده را با git status و git diff بررسی کنید.
9. تست‌های مناسب را اجرا و نتیجهٔ آن‌ها را تفسیر کنید.
10. قبل از commit، تغییرات را از نظر دامنه، امنیت و منطق بررسی کنید.
11. برای کارهای تکراری Skill ایجاد کنید.
12. برای کارهای مستقل و موازی از Subagent یا Worktree استفاده کنید.
13. اتصال ابزارهای بیرونی را با Plugin و MCP ارزیابی کنید.
14. در صورت خراب‌شدن روند کار، نشست را مدیریت و بازیابی کنید.

روش پیشنهادی مطالعه

بهتر است کتابچه را به‌ترتیب فصل‌ها مطالعه کنید، اما برای استفادهٔ روزمره می‌توانید از بخش‌های زیر به‌عنوان مرجع سریع استفاده کنید:

- برای شروع پروژه: فصل سوم
- برای اجرای یک کار استاندارد: فصل چهارم
- برای پیدا کردن دستور مناسب: فصل پنجم
- برای بررسی امنیت و مجوزها: فصل ششم
- برای بررسی قبل از commit: فصل هفتم
- برای تنظیم قوانین پروژه: فصل هشتم
- برای قابلیت‌های پیشرفته: فصل نهم
- برای حل مشکلات رایج: فصل یازدهم

bitwise

پیشنهاد می‌شود هنگام مطالعه، یک پروژهٔ آزمایشی کوچک داشته باشید. پروژهٔ آزمایشی می‌تواند یک API ساده، یک برنامهٔ خط فرمان، یک فرم کوچک، یک پروژهٔ Flutter یا یک سرویس Python باشد. هدف این است که مفاهیم را روی پروژه‌ای اجرا کنید که در صورت خطا، آسیب مهمی ایجاد نکند.

یک اصل مهم: Codex همکار است، نه جایگزین مسئولیت فنی

Codex می‌تواند در خواندن، تحلیل و تغییر کد کمک زیادی کند، اما مسئولیت فنی پروژه را بر عهده نمی‌گیرد. خروجی Codex ممکن است:

- ناقص باشد.
- با معماری پروژه سازگار نباشد.
- یک حالت مرزی را نادیده بگیرد.
- تست کافی نداشته باشد.
- فایل‌های بیشتری از نیاز واقعی تغییر دهد.
- ناخواسته رفتار موجود را تغییر دهد.
- دستور خطرناکی پیشنهاد کند.
- اطلاعات حساس را در خروجی یا diff قرار دهد.

بنابراین، نقش کاربر فقط نوشتن درخواست نیست. کاربر باید هدف را مشخص کند، خروجی را بررسی کند، تست مناسب را اجرا کند و دربارهٔ پذیرش نهایی تغییرات تصمیم بگیرد.

روش کلی این کتابچه

فرایند پیشنهادی کتابچه را می‌توان در شش مرحله خلاصه کرد:

1. **شناخت وضعیت فعلی پروژه**
2. **تعریف هدف و محدودیت‌ها**
3. **برنامه‌ریزی پیش از تغییر**
4. **اجرای کنترل‌شده**
5. **تست و بازبینی**
6. **بررسی نهایی و commit دستی**

این ترتیب ساده، پایهٔ تمام مباحث بعدی است. حتی هنگام استفاده از قابلیت‌های پیشرفته، نباید این مراحل حذف شوند.

دربارهٔ نسخه و تغییرات آینده

Codex، مدل‌های زبانی، محیط‌های اجرا و ابزارهای متصل به آن به‌طور مداوم توسعه پیدا می‌کنند. به همین دلیل، ممکن است یک دستور در نسخه‌ای وجود داشته باشد و در نسخهٔ دیگر تغییر کند، نام یک گزینه تغییر کند یا یک قابلیت فقط در محیط خاصی در دسترس باشد.

برای همین، در سراسر کتابچه از سه نوع عبارت استفاده می‌شود:

- **قاعدهٔ پایدار:** مفهومی که به نسخهٔ خاصی وابستگی زیادی ندارد؛ مانند بررسی git diff قبل از commit.
- **قابلیت نسخه‌محور:** قابلیتی که ممکن است در نسخه‌های مختلف تغییر کند.
- **نمونهٔ پیشنهادی:** دستوری که برای آموزش ارائه شده و باید متناسب با پروژهٔ واقعی تنظیم شود.

پیش از استفاده در محیط تولید، وضعیت نسخهٔ نصب‌شده، مستندات رسمی و سیاست‌های تیم خود را بررسی کنید.

راهنمای استفاده از مثال‌های این کتابچه

در مثال‌های این کتابچه، مسیرها و نام فایل‌ها عمومی هستند. برای نمونه:

```
path/to/file
```

یعنی باید آن را با مسیر واقعی پروژهٔ خود جایگزین کنید؛ برای مثال:

```
src/auth/login.py
```

همچنین عبارت‌های زیر نمونه هستند و باید متناسب با پروژه تغییر کنند:

```
RUN_TESTS
PROJECT_NAME
MODEL
COMMAND
```

نمونهٔ زیر را در نظر بگیرید:

```
pytest tests/test_login.py
```

اگر پروژهٔ شما با ابزار دیگری تست می‌شود، ممکن است دستور مناسب چنین باشد:

```
bash
python -m unittest tests/test_login.py
```

یا:

```
bash
npm test -- login
```

یا:

```
bash
flutter test test/login_test.dart
```

اصل مهم این است که Codex باید دستور واقعی و قابل‌اجرای پروژه را بداند، نه یک دستور فرضی.

bitwithit

فصل اول: آشنایی با Codex

۱.۱. Codex چیست؟

Codex ابزاری مبتنی بر هوش مصنوعی است که می‌تواند در انجام کارهای توسعهٔ نرم‌افزار کمک کند. بسته به محیط و قابلیت‌های در دسترس، Codex ممکن است بتواند فایل‌های پروژه را بخواند، ساختار کد را تحلیل کند، تغییرات پیشنهادی ارائه دهد، فایل‌ها را ویرایش کند، دستورهای موردنیاز را اجرا کند و نتیجه را برای بررسی به کاربر نشان دهد.

تعامل با Codex معمولاً از طریق گفت‌وگو انجام می‌شود؛ اما برخلاف یک گفت‌وگوی عمومی، Codex می‌تواند در برخی محیط‌ها به کدبیس و ابزارهای توسعه دسترسی داشته باشد. همین ویژگی باعث می‌شود که دقت در تعیین پوشهٔ پروژه، سطح دسترسی، هدف کار و محدودیت‌ها اهمیت زیادی داشته باشد.

Codex را بهتر است یک **همکار نرم‌افزاری هوشمند** بدانیم: ابزاری که می‌تواند بخشی از کار را انجام دهد، اما برای تصمیم‌گیری نهایی، بررسی تغییرات و پذیرش نتیجه به نظارت انسان نیاز دارد.

۱.۲. Codex چه مسئله‌ای را حل می‌کند؟

در پروژه‌های نرم‌افزاری، بخش زیادی از زمان توسعه صرف کارهایی می‌شود که تکراری یا چندمرحله‌ای هستند؛ برای مثال:

- پیدا کردن محل پیاده‌سازی یک قابلیت
- بررسی وابستگی میان فایل‌ها
- نوشتن یا اصلاح تست
- رفع خطاهای مشخص
- افزودن یک رفتار جدید به برنامه
- به‌روزرسانی مستندات
- اجرای دستورهای lint و test
- تحلیل خطاهای خروجی تست
- آماده‌سازی تغییرات برای بازبینی

Codex می‌تواند در این کارها سرعت ایجاد کند؛ به‌ویژه وقتی درخواست روشن باشد و محدودهٔ کار به‌درستی تعریف شود. با این حال، سرعت بیشتر به‌تنهایی به معنای کیفیت بیشتر نیست. نتیجه باید با معیارهای پروژه، تست‌ها و بررسی انسانی سنجیده شود.

۱.۳. Codex برای چه کسانی مناسب است؟

Codex می‌تواند برای افراد با سطح‌های مختلف تجربه مفید باشد، اما شیوهٔ استفاده از آن برای هر گروه متفاوت است.

برنامه‌نویسان مبتدی

Codex می‌تواند در توضیح ساختار پروژه، تشریح خطاها و ساخت نمونه‌های ساده کمک کند. برنامه‌نویس مبتدی بهتر است هر تغییر را در مقیاس کوچک انجام دهد و دربارهٔ علت تغییر توضیح بخواهد.

توسعه‌دهندگان حرفه‌ای

توسعه‌دهندگان باتجربه می‌توانند از Codex برای تحلیل کدبیس، ساخت تست، انجام اصلاحات محدود، آماده‌سازی تغییرات و خودکارسازی کارهای تکراری استفاده کنند. در این سطح، مهم است که Codex با استانداردها و معماری واقعی پروژه هماهنگ شود.

تیم‌های نرم‌افزاری

تیم‌ها می‌توانند از فایل‌هایی مانند AGENTS.md برای بیان قواعد پایدار پروژه استفاده کنند. همچنین می‌توانند در صورت سازگاری با نسخه و سیاست سازمان، از محیط‌های جداگانه مانند Worktree یا Cloud برای تقسیم کار بهره ببرند.

دانشجویان و پژوهشگران

Codex می‌تواند به فهم کد، آماده‌سازی آزمایش‌ها، نوشتن تست و مستندسازی کمک کند. در کار پژوهشی، لازم است صحت علمی، reproducibility و منشأ داده‌ها جداگانه بررسی شود؛ خروجی مدل به‌تنهایی شاهد علمی محسوب نمی‌شود.

۱.۴. Codex چه کارهایی می‌تواند انجام دهد؟

قابلیت‌های در دسترس به نسخه، مدل، محیط اجرا و سطح دسترسی بستگی دارند؛ اما به‌طور کلی می‌توان از Codex برای کارهای زیر استفاده کرد.

فهم و مرور کد

- خلاصه‌کردن ساختار یک پروژه
- پیدا کردن فایل‌های مرتبط با یک قابلیت
- توضیح جریان اجرای یک تابع یا ماژول
- بررسی وابستگی میان بخش‌های مختلف سیستم
- توضیح یک خطا یا رفتار غیرمنتظره

تغییر کد

- افزودن قابلیت محدود و مشخص
- اصلاح باگ
- بهبود پیام خطا یا اعتبارسنجی
- بازآرایی محدود کد
- به‌روزرسانی تست‌ها و مستندات

اجرای ابزارهای پروژه

در محیط‌هایی که اجازهٔ اجرای فرمان وجود دارد، Codex ممکن است بتواند دستورهایی مانند تست، lint یا ابزارهای تحلیل ایستا را اجرا کند. دستور مناسب باید از ساختار واقعی پروژه استخراج یا با کاربر تأیید شود.

پشتیبانی از بازبینی

Codex می‌تواند تغییرات را بررسی کند، ریسک‌های احتمالی را گزارش دهد و مواردی را که نیاز به توجه دارند مشخص کند. این بررسی، جایگزین بازبینی انسانی یا اجرای تست نیست.

۱.۵. محدودیت‌ها و مسئولیت کاربر

Codex ممکن است با اطمینان پاسخ دهد، اما اطمینان در بیان به‌تنهایی نشانهٔ درستی نیست. پیش از پذیرش هر تغییر، این محدودیت‌ها را در نظر بگیرید.

احتمال برداشت اشتباه از هدف

اگر درخواست مبهم باشد، Codex ممکن است منظور کاربر را نادرست تفسیر کند. عبارت‌هایی مانند «درستش کن» یا «تمیزترش کن» بدون توضیح بیشتر، محدودهٔ تغییر را روشن نمی‌کنند.

احتمال تغییر فایل‌های غیرضروری

گاهی یک تغییر کوچک می‌تواند Codex را به اصلاح فایل‌های مرتبط یا بازآرایی گسترده‌تر سوق دهد. برای پیشگیری، فایل‌های مجاز و ممنوع را مشخص کنید و پس از اجرا فهرست تغییرات را بررسی کنید.

احتمال خطا در کد یا تست

کد پیشنهادی ممکن است اجرا نشود، با نسخهٔ وابستگی‌ها ناسازگار باشد یا حالتی را پوشش ندهد. تست‌های سبز نیز تضمین نمی‌کنند که همهٔ خطاهای ممکن برطرف شده‌اند؛ آن‌ها فقط نشان می‌دهند آزمون‌های اجراشده موفق بوده‌اند.

احتمال تغییر رفتار موجود

یک اصلاح ظاهراً کوچک می‌تواند رفتار برنامه را در مسیرهای دیگر تغییر دهد. هنگام بازبینی، فقط خطوط جدید را نبینید؛ ارتباط تغییر با رفتارهای موجود را هم در نظر بگیرید.

احتمال افشای اطلاعات حساس

فایل‌های محیطی، کلیدها، توکن‌ها و اطلاعات خصوصی نباید بی‌دلیل در ورودی‌ها، خروجی‌ها یا تغییرات پروژه ظاهر شوند. هر diff را از نظر وجود اطلاعات حساس بررسی کنید.

وابستگی قابلیت‌ها به نسخه و محیط

همهٔ دستورها، مدل‌ها یا روش‌های اجرا در همهٔ نسخه‌ها و محیط‌ها یکسان نیستند. اگر دستوری شناخته‌شده اجرا نشد، وجود آن را در نسخهٔ فعلی بررسی کنید و از دستورهای حدسی برای عبور از محدودیت‌ها استفاده نکنید.

۱.۶. اصول طلایی کار با Codex

اصل اول: هدف را دقیق و محدود بنویسید

به‌جای:

صفحهٔ ورود را بهتر کن.

بنویسید:

در فرم ورود، اگر فیلد ایمیل خالی باشد، پیام اعتبارسنجی مناسب نمایش بده.

فقط فایل‌های مربوط به فرم ورود و تست‌های همان فرم را تغییر بده.

ظاهر سایر بخش‌ها را تغییر نده.

در پایان تست‌های مرتبط را اجرا کن و commit نکن.

اصل دوم: پیش از تغییر، برنامه را بررسی کنید

برای کارهای چندمرحله‌ای، ابتدا از Codex بخواهید فایل‌های مرتبط، مراحل پیشنهادی و ریسک‌ها را توضیح دهد. پس از بررسی برنامه، اجرای تغییر را تأیید کنید.

اصل سوم: سطح دسترسی را متناسب با کار انتخاب کنید

برای شروع یا کارهای ناشناخته، از تنظیمات محدودتر استفاده کنید. دسترسی گسترده را فقط وقتی فعال کنید که بدانید چه کاری انجام می‌شود و چرا به آن دسترسی نیاز است.

اصل چهارم: نتیجه را مستقل بررسی کنید

git status، git diff و خروجی تست را خودتان بررسی کنید. گزارش Codex به‌تنهایی جای مشاهدهٔ تغییر واقعی را نمی‌گیرد.

اصل پنجم: commit را با تصمیم آگاهانه انجام دهید

تا زمانی که تغییرات و تست‌ها را بررسی نکرده‌اید، commit نکنید. در این کتابچه، commit دستی و پس از بررسی انسانی توصیه می‌شود.

فصل دوم: محیط‌های کاری Codex

محیط کاری تعیین می‌کند Codex کجا پروژه را می‌بیند، چگونه به فایل‌ها دسترسی دارد و چه ابزارهایی می‌تواند اجرا کند. نام‌ها و امکانات دقیق ممکن است با نسخه و محصول تفاوت داشته باشند؛ بنابراین این فصل را به‌عنوان راهنمای مفهومی بخوانید و گزینه‌های موجود در محیط خود را نیز بررسی کنید.

۲.۱. اپلیکیشن دسکتاپ

اپلیکیشن دسکتاپ برای کار تعاملی با پروژه مناسب است. در این محیط معمولاً می‌توانید پروژه یا پوشهٔ کاری را انتخاب کنید، با Codex گفت‌وگو کنید، تغییرات را ببینید و در برخی نسخه‌ها از ابزارهای محیطی یا رابط گرافیکی استفاده کنید.

کاربردهای رایج

- شروع کار روی یک پروژهٔ محلی
- درخواست توضیح دربارهٔ کد
- انجام تغییر محدود و مشاهدهٔ diff
- بررسی نتیجه پیش از ادامهٔ کار
- مدیریت چند گفت‌وگو یا نشست کاری، اگر نسخهٔ استفاده‌شده چنین قابلیتی داشته باشد

نکتهٔ مهم

پیش از شروع مطمئن شوید پوشهٔ درست پروژه را انتخاب کرده‌اید. اگر پوشهٔ والد یا پروژهٔ دیگری انتخاب شود، Codex ممکن است زمینهٔ ناقص یا نامرتبطی ببیند.

۲.۲. محیط CLI

CLI یا رابط خط فرمان، راهی برای استفاده از Codex از طریق ترمینال است. این محیط برای توسعه‌دهندگانی مناسب است که بیشتر کار خود را در ترمینال انجام می‌دهند یا می‌خواهند Codex را در گردش‌کارهای متنی و اسکریپتی به کار بگیرند.

کاربردهای رایج

- شروع یک نشست از ریشهٔ پروژه
- انتخاب مدل یا سطح دسترسی، در صورت پشتیبانی نسخه
- اجرای دستورهای مرتبط با پروژه
- استفاده در گردش‌کارهای ترمینالی
- اجرای وظایف غیرتعاملی در قابلیت‌هایی که نسخهٔ نصب‌شده پشتیبانی می‌کند

احتیاط

دستورهای CLI می‌توانند بر اساس سیستم‌عامل، نسخه و تنظیمات نصب متفاوت باشند. پیش از کپی‌کردن یک دستور در محیط حساس، راهنمای نسخهٔ نصب‌شده را بررسی کنید و از اجرای فرمانی که پیامد آن را نمی‌دانید خودداری کنید.

۲.۳. محیط Cloud

در محیط Cloud، بخشی از کار در محیطی میزبانی‌شده انجام می‌شود، نه صرفاً روی رایانهٔ محلی کاربر. این محیط می‌تواند برای کارهای طولانی یا کارهایی که نیاز به اجرای جداگانه دارند مفید باشد، به شرط آنکه در محصول و حساب کاربر در دسترس باشد.

مواردی که باید بررسی شوند

- کدام شاخه یا نسخهٔ پروژه در اختیار محیط قرار می‌گیرد؟
- آیا فایل‌ها یا تنظیمات محلی به محیط Cloud منتقل می‌شوند؟
- چه متغیرهای محیطی در دسترس‌اند؟
- نتیجهٔ کار چگونه به ریپوی محلی بازمی‌گردد؟
- دسترسی‌های شبکه و اجرای فرمان چه محدودیت‌هایی دارند؟
- آیا اطلاعات حساس به‌درستی محافظت می‌شوند؟

به‌دلیل تفاوت تنظیمات سازمانی و محصولی، فرض نکنید محیط Cloud همان فایل‌ها، مجوزها یا متغیرهای محلی را دارد.

۲.۴. Worktree

Worktree قابلیتی در Git است که اجازه می‌دهد چند شاخه از یک ریپو در پوشه‌های کاری جداگانه بررسی یا ویرایش شوند. در ابزارهایی که از Worktree پشتیبانی می‌کنند، این روش می‌تواند برای جداکردن وظایف یا کاهش تداخل تغییرات مفید باشد.

چه زمانی Worktree مناسب است؟

- وقتی می‌خواهید کار آزمایشی را از شاخهٔ اصلی جدا کنید.
- وقتی دو کار مستقل را همزمان پیش می‌برید.
- وقتی می‌خواهید تغییرات یک کار را بدون دست‌کاری پوشهٔ کاری اصلی بررسی کنید.
- وقتی قصد دارید نتیجهٔ کاری را به‌راحتی کنار بگذارید یا با شاخه‌ای دیگر مقایسه کنید.

چه زمانی لازم نیست؟

- برای یک تغییر کوچک و ساده در یک فایل
- وقتی هنوز با مدیریت شاخه‌ها و وضعیت Git آشنا نیستید
- وقتی هر دو کار به فایل‌های یکسان وابستگی جدی دارند

Worktree به‌خودی‌خود کیفیت یا امنیت تغییرات را تضمین نمی‌کند؛ فقط جداسازی محیط کاری را ساده‌تر می‌کند.

۲.۵ .مقایسهٔ محیط‌های اجرا

محیط	مناسب برای	مزیت اصلی	نکتهٔ احتیاطی
اپلیکیشن دسکتاپ	کار تعاملی و بررسی تغییرات	مشاهدهٔ مستقیم پروژه و خروجی	پوشه و سطح دسترسی را پیش از شروع بررسی کنید
CLI	کار ترمینالی و گردش‌کارهای برنامه‌پذیر	ترکیب آسان با ابزارهای خط فرمان	دستورها به نسخه و تنظیمات نصب وابسته‌اند
Cloud	کارهای جداگانه یا طولانی، در صورت پشتیبانی	اجرای کار خارج از محیط محلی	دسترسی فایل‌ها، شبکه و رازها را بررسی کنید
Worktree	کارهای موازی یا جداشده	کاهش تداخل میان شاخه‌ها	مدیریت شاخه‌ها و ادغام تغییرات همچنان لازم است

۲.۶. مرورگر، موبایل و قابلیت صوتی

برخی نسخه‌ها یا محیط‌های کاری ممکن است دسترسی از مرورگر، ادامهٔ کار از دستگاه دیگر یا ورودی صوتی ارائه دهند. جزئیات این قابلیت‌ها نسخه‌محور هستند؛ بنابراین پیش از تکیه‌کردن بر آن‌ها، رفتار دقیق نسخهٔ خود را بررسی کنید.

هنگام استفاده از مرورگر یا موبایل

- مطمئن شوید به همان پروژه یا نشست موردنظر متصل هستید.
- وضعیت آخرین تغییرات را بررسی کنید.
- فرض نکنید دسترسی موبایل معادل اجرای محلی است.
- برای تغییرات مهم، diff را در محیطی مناسب و خوانا بررسی کنید.

هنگام استفاده از ورودی صوتی

ورودی صوتی می‌تواند برای توضیح سریع هدف مفید باشد، اما نام فایل‌ها، مسیرها، نمادهای برنامه‌نویسی و دستورها ممکن است اشتباه تشخیص داده شوند. پس از دیکته‌کردن، متن درخواست را بازخوانی کنید؛ به‌خصوص اگر شامل مسیر، مقدار، نام متغیر یا فرمان حساس است.

قاعدهٔ عملی

صرف‌نظر از اینکه درخواست از طریق دسکتاپ، CLI، مرورگر یا صدا ارسال شده است، نتیجه را با همان معیارهای همیشگی بررسی کنید:

```
bash
git status
git diff
```

سپس تست‌های مرتبط را اجرا کنید.



فصل سوم: راه‌اندازی نخستین پروژه

است. اگر پروژه در پوشهٔ اشتباه باز شود، Codexراه‌اندازی صحیح پروژه، یکی از مهم‌ترین مراحل کار با معرفی نشوند، حتی یک درخواست ساده نیز ممکن است به Codex مشخص نباشد یا قوانین پروژه به Gitوضعیت نتیجهٔ نادرست منجر شود.

هدف این فصل ایجاد یک نقطهٔ شروع قابل‌اعتماد است.

۳.۱. پیش‌نیازها

پیش از شروع کار، موارد زیر را بررسی کنید:

- پروژه در یک پوشهٔ مشخص قرار داشته باشد.
- ریپوی Git به‌درستی شناسایی شود.
- شاخهٔ کاری فعلی را بدانید.
- ابزارهای موردنیاز پروژه نصب باشند.
- دستور اجرای تست یا بررسی کیفیت کد مشخص باشد.
- فایل‌های حساس از دسترس غیرضروری خارج باشند.
- Codex روی پوشهٔ صحیح باز شده باشد.

برای بررسی اولیهٔ Git، از ریشهٔ پروژه دستور زیر را اجرا کنید:

```
bash
git status
```

برای مشاهدهٔ شاخهٔ فعلی:

```
bash
git branch --show-current
```

برای مشاهدهٔ ریشهٔ ریپو:

```
bash
git rev-parse --show-toplevel
```

اگر دستور آخر مسیری غیر از پوشهٔ مورد انتظار نشان داد، پیش از ادامه محیط کاری را اصلاح کنید.

۳.۲. انتخاب ریشهٔ ریپو

بهترین نقطه برای شروع، ریشهٔ ریپوی Git است؛ یعنی پوشه‌ای که معمولاً پوشهٔ .git یا ساختار اصلی پروژه در آن قرار دارد.

برای مثال:

```
my-project/
├── .git/
├── src/
├── tests/
├── README.md
├── pyproject.toml
└── AGENTS.md
```

در این نمونه، باید Codex را روی پوشهٔ my-project باز کنید، نه فقط روی src یا tests.

چرا ریشهٔ ریپو مهم است؟

Codex برای درک پروژه به اطلاعاتی مانند موارد زیر نیاز دارد:

- فایل‌های تنظیمات
- وابستگی‌ها
- تست‌ها
- مستندات
- ساختار ماژول‌ها
- قوانین پروژه
- تاریخچه و وضعیت Git

اگر فقط یک زیرپوشه در اختیار Codex قرار گیرد، ممکن است برخی از این اطلاعات دیده نشوند.

استثنا

در پروژه‌های بسیار بزرگ، یا زمانی که کار کاملاً محدود است، می‌توانید دامنهٔ بررسی را به زیرپوشه‌ای مشخص محدود کنید؛ اما بهتر است به‌صراحت توضیح دهید که:

فقط پوشهٔ src/auth و تست‌های متناظر آن را بررسی کن.

به سایر بخش‌های پروژه دست نزن.

۳.۳Git. بررسی وضعیت اولیهٔ

قبل از شروع هر تغییر، وضعیت فعلی پروژه را ثبت یا بررسی کنید:

```
bash
git status
```

اگر پروژه تمیز باشد، معمولاً پیامی شبیه این می‌بینید:

nothing to commit, working tree clean

اگر فایل‌های تغییرکرده یا جدید وجود داشته باشند، آن‌ها را بشناسید:

```
bash
git status --short
```

نمونهٔ خروجی:

M src/auth/login.py

?? tests/test_login.py

در این خروجی:

- M یعنی فایل تغییر کرده است.
- ?? یعنی فایل جدید و هنوز دنبال‌نشده است.
- A معمولاً به فایل اضافه‌شده اشاره می‌کند.
- D یعنی فایل حذف شده است.

قانون مهم

اگر پیش از شروع کار، تغییرات قبلی در پروژه وجود دارد، آن‌ها را نادیده نگیرید. به Codex بگویید:

پیش از شروع، وضعیت فعلی Git را بررسی کن.

تغییرات موجود را دست‌کاری نکن و فقط روی فایل‌های مرتبط با هدف جدید کار کن.

۳.۴. ایجاد شاخهٔ کاری

برای کارهای مهم یا تغییرات چندمرحله‌ای، ایجاد یک شاخهٔ جداگانه توصیه می‌شود:

```
bash
git switch -c feature/login-validation
```

یا در نسخه‌های قدیمی‌تر Git:

```
bash
git checkout -b feature/login-validation
```

نام شاخه باید کوتاه، واضح و مرتبط با کار باشد. نمونه‌ها:

fix/api-timeout

feature/user-profile

test/payment-service

refactor/image-pipeline

docs/codex-guide

چرا شاخهٔ جداگانه مفید است؟

- تغییرات از شاخهٔ اصلی جدا می‌مانند.
- بازگردانی یا کنارگذاشتن کار ساده‌تر می‌شود.
- بازبینی تغییرات واضح‌تر است.
- چند کار مختلف کمتر با یکدیگر تداخل پیدا می‌کنند.
- امکان ارسال Pull Request یا Merge Request فراهم می‌شود.

نکته

سپرد، اما برای کاربران تازه‌کار بهتر است این کار دستی انجام شود تا Codexایجاد شاخهٔ کاری را می‌توان به شاخهٔ فعلی کاملاً مشخص باشد.

۳.۵. بررسی ابزارها و دستورهای پروژه

هر پروژه روش خاصی برای نصب وابستگی، اجرای برنامه و اجرای تست دارد. پیش از درخواست تغییر، فایل‌های تنظیمات را بشناسید.

نمونه‌هایی از فایل‌های متداول:

کاربرد	فایل
تنظیمات پروژه‌های Python	pyproject.toml
وابستگی‌های Python	requirements.txt
وابستگی و اسکریپت‌های JavaScript	package.json
تنظیمات Flutter و Dart	pubspec.yaml
پروژه‌های Rust	Cargo.toml
پروژه‌های Go	go.mod
فرمان‌های استاندارد پروژه	Makefile
ساخت image کانتینر	Dockerfile
راهنمای نصب و اجرا	README.md

از Codex بخواهید ابتدا دستورهای اصلی پروژه را پیدا کند:

ساختار پروژه و فایل‌های تنظیمات را بررسی کن.

دستورهای نصب وابستگی، اجرای برنامه، اجرای تست و lint را شناسایی کن.

فعلاً هیچ فایلی را تغییر نده.

این درخواست فقط مرحلهٔ شناخت است و از اجرای تغییر زودهنگام جلوگیری می‌کند.

۳.۶. ایجاد و بررسی AGENTS.md

فایل AGENTS.md برای بیان قوانین و ترجیحات پروژه به Codex استفاده می‌شود. این فایل باید کوتاه، دقیق و اجرایی باشد.

این درخواست چند مزیت دارد:

- پروژه را به Codex معرفی می‌کند.
- میزان درک Codex از ساختار را نشان می‌دهد.
- خطاهای مسیر یا محیط را زود آشکار می‌کند.
- بدون ایجاد تغییر، زمینهٔ لازم را فراهم می‌کند.

پس از دریافت پاسخ، آن را با ساختار واقعی پروژه مقایسه کنید.

۳.۸. درخواست اصلاح محدود

پس از شناخت پروژه، یک تغییر کوچک تعریف کنید:

در فایلsrc/auth/login.py ، پیام خطای مربوط به ایمیل نامعتبر را واضح‌تر کن.

فقط همین فایل و تست مرتبط را تغییر بده.

اگر تست مرتبط وجود ندارد، ابتدا پیشنهاد بده و بدون تأیید من تست جدید ایجاد نکن.

commitو push انجام نده.

این نوع درخواست از چند بخش مهم تشکیل شده است:

1. **هدف:** واضح‌ترکردن پیام خطا
2. **محدوده:** دو فایل مشخص
3. **شرط:** اگر تست وجود نداشت، فقط پیشنهاد ارائه شود
4. **ممنوعیت:** commit و push انجام نشود

هرچه این بخش‌ها دقیق‌تر باشند، کنترل نتیجه آسان‌تر خواهد بود.

فصل چهارم: چرخهٔ استاندارد انجام کار

کار با Codex بهتر است به‌صورت یک چرخهٔ مشخص انجام شود، نه مجموعه‌ای از درخواست‌های جدا و بی‌برنامه.

چرخهٔ پیشنهادی:

شناخت

↓

تعریف هدف

↓

برنامه‌ریزی

↓

اجرا

↓

تست

↓

بازبینی

↓

commit تصمیم برای

هیچ مرحله‌ای نباید بدون دلیل حذف شود. برای تغییرات کوچک، برخی مراحل کوتاه‌تر می‌شوند؛ اما اصل آن‌ها همچنان باقی می‌ماند.

۴.۱. مرحلهٔ اول: شناخت وضعیت فعلی

پیش از هر تغییری، وضعیت پروژه را بررسی کنید:

git status

سپس از Codex بخواهید فایل‌های مرتبط را شناسایی کند

ابتدا بدون تغییر فایل‌ها، محل پیاده‌سازی قابلیت موردنظر را پیدا کن.

فایل‌های مرتبط، تست‌های موجود و وابستگی‌های مهم را فهرست کن.

در این مرحله فقط تحلیل ارائه بده.

خروجی مورد انتظار

Codex باید بتواند موارد زیر را مشخص کند:

- فایل اصلی موردنظر
- فایل‌های وابسته
- تست‌های مرتبط
- قراردادها یا interfaceهای مؤثر

- خطرهای احتمالی
- دستورهای لازم برای اعتبارسنجی

اگر پاسخ کلی یا مبهم بود، پیش از اجرای تغییر سؤال‌های دقیق‌تری بپرسید.

۴.۲. مرحلهٔ دوم: تعریف هدف

هدف باید شامل نتیجهٔ مورد انتظار و محدودیت‌های کار باشد.

الگوی پیشنهادی

هدف:

[تغییر موردنظر]

محدوده:

[فایل‌ها یا پوشه‌های مجاز]

خارج از محدوده:

[بخش‌هایی که نباید تغییر کنند]

معیار پذیرش:

[چطور بفهمیم کار درست انجام شده است؟]

تست:

[چه تستی باید اجرا شود؟]

محدودیت:

[چه کارهایی نباید انجام شود؟]

۴.۳. مرحلهٔ سوم: برنامه‌ریزی با /plan

برای کارهای چندمرحله‌ای، از Codex بخواهید برنامه ارائه دهد. در محیط‌هایی که دستور /plan پشتیبانی می‌شود، می‌توانید از آن استفاده کنید:

```
/plan
```

یا در متن درخواست:

برای این کار یک برنامهٔ مرحله‌به‌مرحله ارائه بده.

فعلاً هیچ فایلی را تغییر نده.

برای هر مرحله فایل‌های درگیر، ریسک و تست پیشنهادی را بنویس.

یک برنامهٔ خوب باید شامل این موارد باشد

- هدف کلی
- فایل‌های احتمالی
- ترتیب مراحل
- وابستگی میان مراحل
- تست‌های لازم
- ریسک‌های احتمالی
- تصمیم‌هایی که نیاز به تأیید کاربر دارند

نمونهٔ برنامه

۱ .بررسی route مربوط به پروفایل.

۲ .بررسی middleware احراز هویت.

۳ .بررسی تست‌های موجود برای کاربر ناشناس.

۴ .افزودن یا اصلاح رفتار پاسخ 401.

۵ .اضافه‌کردن تست در صورت نبود پوشش کافی.

۶ .اجرای تست‌های API مربوط.

۷ .بررسی diff و اطمینان از عدم تغییرendpoint های دیگر.

پس از بررسی برنامه، اگر با آن موافق بودید، اجرای آن را درخواست کنید:

برنامه تأیید است. آن را اجرا کن.

فقط دامنهٔ تعیین‌شده را تغییر بده و پس از پایان، تست‌ها و خلاصهٔ فایل‌های تغییرکرده را گزارش کن.

۴.۴. مرحلهٔ چهارم: اجرای کنترل‌شده

هنگام اجرای تغییر، درخواست را به یک هدف مشخص محدود نگه دارید. اگر کار بزرگ است، آن را به چند مرحله تقسیم کنید.

به‌جای:

کل سیستم احراز هویت را بازنویسی کن.

از چند درخواست کوچک استفاده کنید:

مرحلهٔ اول:

فقط ساختار فعلی احراز هویت را بررسی کن و نقاط ضعف احتمالی را گزارش بده.

هیچ فایلی را تغییر نده.

سپس:

مرحلهٔ دوم:

فقط اعتبارسنجی ورودی endpoint login را اصلاح کن.

فایل‌های مجاز... :

تست‌های مجاز... :

مزایای تقسیم کار

- تشخیص خطا آسان‌تر می‌شود.
- diff کوچک‌تر می‌ماند.
- امکان بازگشت به مرحلهٔ قبل وجود دارد.
- احتمال تغییر ناخواسته کاهش می‌یابد.
- بررسی هر مرحله مستقل انجام می‌شود.

۴.۵. مرحلهٔ پنجم: اجرای تست

پس از تغییر، تست مرتبط را اجرا کنید. دستور تست باید از پروژهٔ واقعی گرفته شود.

نمونه‌ها:

```
bash
pytest tests/test_auth.py
bash
npm test -- --runInBand
bash
flutter test test/auth_test.dart
bash
go test ./...
```

از Codex بخواهید ابتدا دستور مناسب را از فایل‌های پروژه شناسایی کند، نه اینکه فرمانی فرضی بسازد.

هنگام شکست تست چه کنیم؟

اگر تست شکست خورد، از Codex بخواهید:

خروجی کامل تست را تحلیل کن.

علت شکست را مشخص کن.

ابتدا فقط علت و راهکارهای پیشنهادی را ارائه بده و هنوز فایل‌ها را تغییر نده.

پس از فهم علت، اصلاح را در دامنه‌ای مشخص درخواست کنید.

نکتهٔ مهم

تغییر کد فقط برای سبزشدن تست، بدون فهم علت شکست، روش قابل‌اعتمادی نیست. گاهی تست قدیمی است، گاهی محیط ناقص است و گاهی رفتار جدید با انتظار قبلی تعارض دارد.

۴.۶. مرحلهٔ ششم: بررسی وضعیت و diff

پس از تغییر، این دو دستور را اجرا کنید:

```
Bash
git status
```

و:

```
bash
git diff
```

برای مشاهدهٔ خلاصهٔ تغییرات:

```
Bash
git diff --stat
```

برای بررسی فایل‌های staged:

```
bash
git diff --cached
```

پرسش‌های ضروری هنگام بررسی diff

- آیا فقط فایل‌های مورد انتظار تغییر کرده‌اند؟
- آیا فایل جدیدی بدون دلیل ایجاد شده است؟
- آیا فایل یا بخشی حذف شده است؟
- آیا قالب‌بندی غیرضروری کل فایل را تغییر داده است؟
- آیا نام، مسیر یا تنظیمات حساسی در diff دیده می‌شود؟
- آیا تست مناسب اضافه یا به‌روزرسانی شده است؟

- آیا منطق قبلی ناخواسته تغییر کرده است؟
- آیا پیام‌های خطا و رفتارهای مرزی بررسی شده‌اند؟

۴.۷. مرحلهٔ هفتم: بازبینی با /review

در محیط‌هایی که دستور /review وجود دارد، می‌توانید از آن برای بازبینی تغییرات استفاده کنید:

```
/review
```

یا:

تغییرات فعلی را مانند یک بازبین کد بررسی کن

تمرکز بر باگ منطقی، شکست رفتار قبلی، نبود تست، مسائل امنیتی و تغییرات خارج از محدوده باشد.

فعلاً کد را تغییر نده و فقط گزارش بازبینی ارائه کن.

قالب مناسب گزارش بازبینی

از Codex بخواهید یافته‌ها را بر اساس اهمیت دسته‌بندی کند:

گزارش را در این قالب ارائه کن:

-بحرانی:

-مهم:

-متوسط:

-پیشنهادی:

-موارد بدون مشکل:

همچنین بخواهید برای هر مورد، محل دقیق آن را مشخص کند:

برای هر یافته، نام فایل، شمارهٔ خط تقریبی، دلیل مشکل و پیشنهاد اصلاح را بنویس.

بازبینی مستقل

اگر امکان استفاده از نشست، مدل یا عامل جداگانه وجود دارد، بازبینی را در محیطی مستقل انجام دهید. هدف این است که تحلیل بازبینی فقط تکرار فرضیات مرحلهٔ پیاده‌سازی نباشد.

۴.۸. مرحلهٔ هشتم: تصمیم دربارهٔ commit

پس از تکمیل مراحل قبلی، سه وضعیت ممکن است وجود داشته باشد:

وضعیت اول: تغییر آمادهٔ commit است

شرایط معمول:

- دامنهٔ تغییر درست است.
- تست‌ها موفق‌اند.
- diff بررسی شده است.
- اطلاعات حساس وجود ندارد.
- بازبینی مشکل مهمی گزارش نکرده است.

در این حالت می‌توانید commit را دستی انجام دهید:

```
bash
git add path/to/file
git commit -m "Fix login validation"
```

وضعیت دوم: تغییر نیاز به اصلاح دارد

اگر بازبینی یا تست مشکل نشان داد، ابتدا مشکل را مشخص کنید:

این یافته را اصلاح کن:

[شرح دقیق یافته]

.فقط فایل‌های مرتبط با همین مشکل را تغییر بده

.پس از اصلاح، تست مربوط را دوباره اجرا کن

وضعیت سوم: تغییر باید کنار گذاشته شود

اگر نتیجه مناسب نیست، بدون بررسی بیشتر آن را commit نکنید. می‌توانید تغییرات را دستی بازگردانید یا از ابزارهای Git با احتیاط استفاده کنید.

از اجرای کورکورانهٔ دستورهای مخرب خودداری کنید؛ به‌ویژه:

```
bash
git reset --hard
```

این دستور می‌تواند تغییرات محلی را برای همیشه حذف کند.

۴.۹. چرخهٔ کامل در قالب یک الگوی آماده

ابتدا وضعیت پروژه و شاخهٔ فعلی را بررسی کن.

تغییرات موجود را دست‌کاری نکن.

هدف:

[توضیح دقیق هدف]

فایل‌های مجاز:

[فهرست فایل‌ها]

فایل‌های غیرمجاز:

[فهرست فایل‌ها یا بخش‌ها]

معیار پذیرش:

[شرایط موفقیت]

ابتدا یک برنامهٔ کوتاه ارائه بده و فعلاً تغییری ایجاد نکن.

پس از تأیید من، تغییر را اجرا کن.

بعد از اجرا:

1. تست‌های مرتبط را اجرا کن.
2. فایل‌های تغییرکرده را فهرست کن.
3. خلاصهٔ diff را ارائه بده.
4. خطاها و محدودیت‌های باقی‌مانده را گزارش کن.
5. commit و push انجام نده.

۴.۱۰. خلاصهٔ فصل چهارم

چرخهٔ استاندارد کار با Codex چنین است:

۱. وضعیت پروژه را بشناس.

۲ .هدف را دقیق بنویس.

۳ .دامنه و محدودیت را مشخص کن.

۴ .برای کار پیچیده برنامه ارائه بده.

۵ .تغییر را مرحله‌ای اجرا کن.

۶ .تست مرتبط را اجرا کن.

۷ status .و diff را بررسی کن.

۸ .بازبینی انجام بده.

۹ .سپس دربارهٔ commit تصمیم بگیر.

این چرخه، حتی در پروژه‌های کوچک، احتمال خطا و تغییر ناخواسته را به‌طور چشمگیری کاهش می‌دهد.

فصل پنجم: راهنمای دستورهای اسلش و CLI

دستورهای اسلش و فرمان‌های CLI دو ابزار مهم برای مدیریت کار با Codex هستند. این دستورها به شما کمک می‌کنند نشست را کنترل کنید، هدف را مشخص کنید، برنامه بگیرید، مدل را تغییر دهید، وضعیت محیط را بررسی کنید و در صورت نیاز گفت‌وگو را کوتاه یا از نو آغاز کنید.

نکتهٔ نسخه‌ای: نام، تعداد و رفتار برخی دستورها ممکن است در نسخه‌های مختلف Codex متفاوت باشد. اگر دستوری در محیط شما شناسایی نشد، فهرست راهنمای همان نسخه را بررسی کنید و از جایگزین‌های حدسی استفاده نکنید.

۵.۱. تفاوت دستور اسلش و دستور CLI

دستور اسلش چیست؟

دستور اسلش در داخل گفت‌وگوی Codex وارد می‌شود و معمولاً برای کنترل نشست یا درخواست عملیات داخلی استفاده می‌شود.

نمونه:

```
/plan
```

یا:

```
/review
```

این دستورها معمولاً بخشی از متن عادی درخواست نیستند، بلکه به محیط می‌گویند یک حالت یا عملیات خاص را فعال کند.

دستور CLI چیست؟

دستور CLI در ترمینال سیستم‌عامل اجرا می‌شود؛ برای مثال:

```
bash
```

```
git status
```

یا:

```
bash
pytest tests/
```

این فرمان‌ها مستقیماً به سیستم‌عامل، Git یا ابزارهای پروژه مربوط هستند.

تفاوت عملی

نوع	محل اجرا	نمونه	کاربرد
دستور اسلش	داخل نشست Codex	/plan	برنامه‌ریزی یا کنترل نشست
دستور CLI	ترمینال	git status	بررسی پروژه یا اجرای ابزار
دستور پروژه	ترمینال یا محیط اجرا	pytest	تست و اعتبارسنجی
دستور Git	ترمینال یا محیط مجاز	git diff	مشاهدهٔ تغییرات

۵.۲. دستورهای هدف و برنامه‌ریزی

```
/goal
```

برای تعریف یا تمرکز روی هدف اصلی کار استفاده می‌شود، در محیط‌هایی که این دستور را پشتیبانی می‌کنند.

نمونه:

```
/goal
```

سپس هدف را به‌صورت مشخص بنویسید:

افزودن اعتبارسنجی شمارهٔ تلفن در فرم ثبت‌نام،

بدون تغییر درAPI های موجود و همراه با تست واحد.

هدف خوب چه ویژگی‌هایی دارد؟

هدف باید:

- نتیجهٔ مورد انتظار را بیان کند.
- محدودهٔ تغییر را مشخص کند.
- محدودیت‌های مهم را بگوید.
- معیار پذیرش داشته باشد.
- از عبارت‌های مبهم دور باشد.

نمونهٔ ضعیف:

این بخش را بهتر کن.

نمونهٔ بهتر:

اعتبارسنجی ورودی فرم ثبت‌نام را برای شمارهٔ تلفن اصلاح کن.

فقط فایل فرم و تست‌های مرتبط را تغییر بده.

فرمت فعلی پیام‌های خطا را حفظ کن.

/plan

برای دریافت برنامهٔ مرحله‌به‌مرحله استفاده می‌شود:

```
/plan
```

درخواست برنامه باید مشخص کند که فعلاً تغییری ایجاد نشود:

برای افزودن قابلیت جست‌وجوی کاربر برنامه‌ریزی کن.

فایل‌های احتمالی، وابستگی‌ها، ریسک‌ها و تست‌های لازم را مشخص کن.

فعلاً هیچ فایلی را تغییر نده.

زمان مناسب استفاده از /plan

- تغییر چندفایلی
- تغییر معماری
- اصلاح باگ پیچیده
- مهاجرت نسخهٔ وابستگی
- تغییر در پایگاه داده
- کارهایی که نیاز به تصمیم‌گیری دارند
- وظایفی که احتمال تغییر خارج از محدوده در آن‌ها زیاد است

برای تغییرهای بسیار کوچک، برنامه می‌تواند کوتاه و یک‌مرحله‌ای باشد.

۵.۳. دستورهای وضعیت و بازبینی

/review

برای درخواست بازبینی تغییرات فعلی استفاده می‌شود:

```
/review
```

درخواست کامل‌تر:

تغییرات فعلی را از نظر باگ منطقی، شکست رفتار قبلی، امنیت،

کامل‌بودن تست‌ها و تغییرات خارج از محدوده بررسی کن.

کد را تغییر نده و فقط گزارش ارائه بده.

/diff

اگر در نسخهٔ محیط شما پشتیبانی شود، می‌تواند برای تمرکز روی تغییرات یا نمایش diff به کار رود. بااین‌حال، دستور قطعی و قابل‌اعتماد برای بررسی تغییرات Git این است:

bash
git diff

برای مشاهدهٔ فایل‌های تغییرکرده:

bash
git status --short
/status

در برخی محیط‌ها برای مشاهدهٔ وضعیت نشست، پروژه یا عملیات جاری استفاده می‌شود. این دستور ممکن است با وضعیت Git یکی نباشد.

بنابراین میان این دو تفاوت بگذارید:

وضعیت نشستCodex

و:

bash
git status

اولی وضعیت ابزار و نشست را نشان می‌دهد؛ دومی وضعیت فایل‌ها و شاخهٔ Git را.

/help

برای مشاهدهٔ دستورهای در دسترس:

/help

این دستور یکی از مهم‌ترین دستورها در شروع کار با یک نسخهٔ جدید است. پیش از حفظکردن فهرست دستورها، بهتر است راهنمای محیط فعلی را بررسی کنید.

۵.۴. دستورهای مدل و استدلال

نام دقیق این گزینه‌ها ممکن است با نسخهٔ Codex یا رابط مورد استفاده متفاوت باشد، اما معمولاً ابزارهایی برای انتخاب مدل و تنظیم میزان استدلال وجود دارد.

انتخاب مدل

ممکن است بتوانید مدل را از طریق رابط یا دستور مربوط به مدل تغییر دهید. هنگام انتخاب مدل، این عوامل را در نظر بگیرید:

- پیچیدگی کار
- مقدار زمینهٔ لازم
- حساسیت پروژه
- زمان قابل‌قبول
- هزینه یا سهمیه
- نیاز به تحلیل عمیق
- اهمیت دقت در برابر سرعت

Reasoning Effort

Reasoning Effort یا میزان تلاش استدلالی، به‌صورت مفهومی نشان می‌دهد مدل چه مقدار منابع و زمان برای تحلیل مسئله اختصاص دهد.

سطوح متداول ممکن است شامل این موارد باشند:

Low

Medium

High

نام دقیق سطوح ممکن است متفاوت باشد.

انتخاب سطح مناسب

نوع کار	سطح پیشنهادی
توضیح یک تابع ساده	کم
تغییر محدود و مشخص	کم تا متوسط
رفع باگ چندفایلی	متوسط
تغییر معماری	زیاد
تحلیل امنیتی یا مهاجرت حساس	زیاد
تولید مستندات ساده	کم
بررسی رفتارهای مرزی پیچیده	زیاد

قاعدهٔ مهم

بالابردن Reasoning Effort همیشه نتیجهٔ بهتر ایجاد نمی‌کند. اگر درخواست مبهم باشد، استدلال بیشتر ممکن است فقط به تحلیل طولانی‌تر و خروجی گسترده‌تر منجر شود. ابتدا هدف را دقیق کنید، سپس در صورت نیاز عمق استدلال را افزایش دهید.

۵.۵. دستورهای محیط اجرا

برخی محیط‌ها گزینه‌هایی برای تغییر یا مشاهدهٔ محیط اجرا دارند؛ مانند:

- اجرای محلی
- محیط ایزوله
- محیط ابری
- Worktree
- حالت فقط‌خواندنی
- حالت اجازه‌دار برای تغییر فایل
- حالت نیازمند تأیید برای اجرای فرمان

این گزینه‌ها را باید بر اساس نوع کار انتخاب کنید.

برای تحلیل فقط‌خواندنی

فقط فایل‌ها را بررسی کن.

هیچ فایلی را تغییر نده و هیچ فرمانی که داده‌ها را تغییر می‌دهد اجرا نکن.

برای تغییر محدود

فقط فایل‌های مشخص‌شده را تغییر بده.

برای اجرای فرمان‌های خارج از تست‌های پروژه، ابتدا تأیید بگیر.

برای کار آزمایشی

این کار را در محیط جداگانه انجام بده.

به شاخهٔ اصلی و فایل‌های خارج از محدوده دست نزن.

اصل انتخاب محیط

هرچه پیامد خطا بیشتر باشد، محیط باید محدودتر و قابل‌بازگشت‌تر باشد.

۵.۶. دستورهای مدیریت نشست

/compact

برای کوتاه‌کردن زمینهٔ گفت‌وگو و حفظ خلاصهٔ اطلاعات مهم استفاده می‌شود؛ در نسخه‌هایی که پشتیبانی می‌کنند.

این دستور زمانی مفید است که:

- گفت‌وگو طولانی شده باشد.
- پاسخ‌ها شروع به تکرار کنند.
- Codex جزئیات مهم را در میان پیام‌های زیاد گم کند.

- می‌خواهید همان کار را با زمینهٔ فشرده ادامه دهید.

پس از compact، موارد مهم را دوباره کنترل کنید:

برای اطمینان، هدف، فایل‌های مجاز، فایل‌های ممنوع و معیار پذیرش فعلی را خلاصه کن.

/new

برای شروع نشست جدید استفاده می‌شود، در صورت پشتیبانی محیط.

زمان مناسب:

- هدف پروژه عوض شده است.
- گفت‌وگو با فرض‌های قدیمی آلوده شده است.
- پاسخ‌ها دیگر به کار فعلی مربوط نیستند.
- می‌خواهید کار جدید را از زمینهٔ قبلی جدا کنید.

پیش از شروع نشست جدید، اطلاعات ضروری را در یک پیام کوتاه ثبت کنید:

پروژه:

شاخه:

هدف:

فایل‌های مجاز:

وضعیت فعلی تست:

محدودیت‌ها:

/side

در برخی محیط‌ها برای ایجاد یا بازکردن گفت‌وگوی جانبی استفاده می‌شود. این قابلیت برای بررسی یک سؤال فرعی بدون برهم‌زدن جریان اصلی مفید است.

مثال:

- گفت‌وگوی اصلی: پیاده‌سازی قابلیت
- گفت‌وگوی جانبی: بررسی یک روش جایگزین
- گفت‌وگوی جانبی: تحلیل یک خطای تست
- گفت‌وگوی جانبی: بررسی ریسک امنیتی

پس از پایان گفت‌وگوی جانبی، نتیجهٔ مهم آن را به جریان اصلی منتقل کنید؛ اما تمام خروجی جانبی را بدون بررسی وارد کار اصلی نکنید.

پایان‌دادن به نشست

وقتی هدف کامل شد، نشست را صرفاً به‌دلیل طولانی‌شدن ادامه ندهید. ابتدا نتیجه را ثبت کنید:

هدف انجام‌شده:

فایل‌های تغییرکرده:

تست‌های اجراشده:

مسائل باقی‌مانده:

وضعیت:commit

۵.۷. دستورهای ابزار و حافظه

در برخی محیط‌ها، Codex می‌تواند از ابزارهای متصل، فایل‌های راهنما یا زمینه‌های ذخیره‌شده استفاده کند. مهم است بدانید اطلاعات از کجا آمده‌اند.

منابع زمینه

زمینهٔ پاسخ ممکن است از این منابع تشکیل شده باشد:

- متن گفت‌وگو
- فایل‌های پروژه
- AGENTS.md
- خروجی فرمان‌ها
- مستندات محلی
- ابزارهای متصل
- نتایج تست
- اطلاعات ذخیره‌شدهٔ نشست

قانون منبع‌یابی

اگر پاسخ به تصمیم فنی مهم منجر می‌شود، از Codex بخواهید منبع خود را مشخص کند:

این نتیجه را بر اساس کدام فایل‌ها و شواهد پروژه ارائه کردی؟

نام فایل و بخش مرتبط را بنویس.

حافظه و زمینهٔ قدیمی

اگر اطلاعات قدیمی ممکن است با وضعیت فعلی پروژه تعارض داشته باشد، اطلاعات فعلی را معیار قرار دهید:

وضعیت فایل‌های فعلی و Git را بر هر فرض یا اطلاعات قبلی مقدم بدان.

۵.۸. دستورهای رایج CLI برای کار با پروژه

وضعیت Git

```bash
git status
```

شاخهٔ فعلی

```bash
git branch --show-current
```

نمایش شاخه‌ها

```bash
git branch
```

مشاهدهٔ تغییرات unstaged

```bash
git diff
```

مشاهدهٔ خلاصهٔ تغییرات

```bash
git diff --stat
```

مشاهدهٔ فایل‌های تغییرکرده

```bash
git status --short
```

مشاهدهٔ تاریخچهٔ کوتاه

```bash
git log --oneline -n 10
```

مشاهدهٔ ریشهٔ ریپو

```bash
git rev-parse --show-toplevel
```

جست‌وجوی متن در پروژه

اگر ripgrep نصب باشد:

```bash
rg "search_term"
```

یا محدود به یک مسیر:

```bash
rg "login" src tests
```

یافتن فایل‌ها

```bash
find . -name "*.py"
```

در ویندوز یا محیط‌هایی که find متفاوت است، از ابزار مناسب همان سیستم استفاده کنید.

بررسی نسخهٔ ابزار

bash

python --version

bash

node --version

bash

git --version

اجرای تست‌های Python

```
bash
pytest
```

یا:

```
bash
python -m pytest
```

اجرای تست‌های JavaScript

```
bash
npm test
```

اجرای تست Flutter

```
bash
flutter test
```

این فرمان‌ها نمونه هستند. همیشه دستور واقعی تعریف‌شده در پروژه را مقدم بدانید.

فصل ششم: کنترل دسترسی و امنیت

کار با Codex زمانی ارزشمند است که سرعت بیشتر با کنترل امنیتی همراه باشد. دادن دسترسی گسترده به یک عامل هوشمند بدون تعیین محدوده می‌تواند خطرهایی مانند حذف فایل، تغییر تنظیمات، افشای رازها یا اجرای فرمان ناخواسته ایجاد کند.

امنیت در این کتابچه بر چهار اصل استوار است:

کمترین دسترسی لازم

+

محدودهٔ روشن

+

تأیید عملیات حساس

+

بررسی مستقل نتیجه

۶.۱. Sandbox چیست؟

Sandbox یا محیط ایزوله، محیطی است که دسترسی Codex را به فایل‌ها، شبکه، فرآیندها یا عملیات سیستم محدود می‌کند.

هدف Sandbox این است که اگر یک فرمان اشتباه یا ناخواسته اجرا شد، دامنهٔ آسیب محدود بماند.

محدودیت‌های احتمالی Sandbox

بسته به محیط، Sandbox ممکن است دسترسی به این موارد را محدود کند:

- فایل‌های خارج از پروژه
- شبکه و اینترنت
- اجرای برنامه‌های خاص
- نوشتن در مسیرهای سیستمی
- دسترسی به credentialها
- فرآیندهای در حال اجرا
- منابع سخت‌افزاری

اصل مهم

Sandbox جایگزین بررسی کاربر نیست. محیط ایزوله می‌تواند ریسک را کاهش دهد، اما اگر دسترسی پروژهٔ اشتباه انتخاب شده باشد، همچنان تغییرات ناخواسته ممکن است رخ دهد.

۶.۲. Approval Policy چیست؟

Approval Policy یا سیاست تأیید مشخص می‌کند Codex برای چه عملیاتی باید از کاربر اجازه بگیرد.

عملیات نیازمند تأیید ممکن است شامل موارد زیر باشد:

- اجرای فرمان جدید
- دسترسی به شبکه
- تغییر فایل خارج از محدوده

- حذف فایل
- نصب وابستگی
- اجرای ابزار پرریسک
- تغییر تنظیمات سیستم
- اجرای فرمانی با پیامد غیرقابل‌بازگشت

چرا تأیید مهم است؟

تأیید یک نقطهٔ توقف ایجاد می‌کند. در این نقطه، کاربر می‌تواند بپرسد:

- این فرمان دقیقاً چه می‌کند؟
- روی کدام فایل‌ها اثر می‌گذارد؟
- آیا تغییرات قابل‌بازگشت هستند؟
- آیا فرمان به شبکه یا اطلاعات حساس دسترسی دارد؟
- آیا روش کم‌خطرتری وجود دارد؟

هنگام دریافت درخواست تأیید

فقط بر اساس نام فرمان تصمیم نگیرید. برای مثال، یک اسکریپت ساده ممکن است در ظاهر بی‌خطر باشد، اما فایل‌های زیادی را تغییر دهد. مسیرها، آرگومان‌ها و اثر جانبی فرمان را بررسی کنید.

۶.۳. سطوح دسترسی متداول

نام دقیق حالت‌ها ممکن است متفاوت باشد، اما از نظر مفهومی می‌توان آن‌ها را چنین دسته‌بندی کرد:

حالت فقط‌خواندنی

در این حالت، Codex برای تحلیل و توضیح مناسب است.

کاربرد:

- آشنایی با پروژه
- تحلیل خطا
- تهیهٔ برنامه
- بررسی معماری
- بازبینی اولیه

حالت ویرایش محدود

Codex می‌تواند فایل‌های مشخصی را تغییر دهد.

کاربرد:

- اصلاح باگ محدود
- افزودن تست
- ویرایش مستندات
- تغییر یک ماژول مشخص

حالت اجرای کنترل‌شده

Codex می‌تواند برخی دستورهای پروژه را اجرا کند، اما عملیات حساس نیازمند تأیید است.

کاربرد:

- اجرای تست
- lint
- type-check
- ساخت پروژه
- بررسی خروجی ابزارها

حالت دسترسی گسترده

این حالت باید با احتیاط استفاده شود و برای کارهای حساس یا ناشناخته مناسب نیست. پیش از فعال‌سازی، دقیقاً بدانید:

- چه فایل‌هایی قابل تغییرند.
- شبکه در دسترس است یا نه.
- فرمان‌ها بدون تأیید اجرا می‌شوند یا نه.
- داده‌های محلی چه مقدار در اختیار محیط قرار می‌گیرند.

۶.۴. Computer Use

برخی محیط‌ها ممکن است قابلیت تعامل با رابط گرافیکی یا رایانه را ارائه دهند. این قابلیت‌ها می‌توانند برای کارهایی مانند بازکردن ابزار، مشاهدهٔ خروجی یا تعامل با برنامه‌های گرافیکی مفید باشند؛ اما سطح ریسک آن‌ها از ویرایش متن ساده بیشتر است.

هنگام استفاده از Computer Use

- پنجرهٔ هدف را بررسی کنید.
- اطلاعات شخصی را روی صفحه نمایش ندهید.
- قبل از کلیک‌های حساس، هدف عملیات را تأیید کنید.
- دانلود، حذف، ارسال یا انتشار اطلاعات را بدون بررسی انجام ندهید.
- پس از پایان، تغییرات ایجادشده را بررسی کنید.

عملیات حساس در رابط گرافیکی

عملیات زیر نیازمند دقت ویژه هستند:

- ارسال فرم
- حذف داده
- انتشار نسخه
- ارسال پیام
- تغییر تنظیمات حساب
- نصب نرم‌افزار
- بارگذاری فایل
- تغییر مجوزها

در این موارد، تأیید نهایی کاربر باید حفظ شود.

۶.۵. عملیات پرریسک

برخی دستورها و عملیات باید همیشه با احتیاط اجرا شوند.

حذف فایل یا پوشه

```
bash
rm -rf path/to/folder
```

این فرمان می‌تواند داده‌ها را بدون امکان بازگردانی آسان حذف کند. Codex نباید چنین عملیاتی را بدون درخواست صریح و بررسی دقیق انجام دهد.

بازنشانی سخت Git

```
bash
git reset --hard
```

این دستور تغییرات محلی ثبت‌نشده را حذف می‌کند.

حذف فایل‌های دنبال‌نشده

```
bash
git clean -fd
```

این فرمان می‌تواند فایل‌های جدیدی را که هنوز commit نشده‌اند حذف کند.

Push اجباری

```
bash
git push --force
```

این عملیات ممکن است تاریخچهٔ شاخهٔ دوردست را بازنویسی کند و کار دیگران را تحت‌تأثیر قرار دهد.

نصب وابستگی از منبع ناشناخته

نصب یک package ناشناخته یا اجرای اسکریپت نصب بدون بررسی می‌تواند خطر امنیتی ایجاد کند.

اجرای اسکریپت ناشناخته

پیش از اجرای اسکریپت، محتوای آن، منبع آن و سطح دسترسی موردنیاز را بررسی کنید.

۶.۶. مدیریت فایل‌های حساس

فایل‌ها و داده‌های زیر باید در اولویت حفاظت باشند:

.env

.env.*

credentials

secrets

private keys

API tokens

SSH keys

service-account files

database dumps

personal data

قوانین پیشنهادی

- فایل‌های حساس را در prompt کپی نکنید.
- مقدار واقعی کلیدها و توکن‌ها را نمایش ندهید.
- فایل‌های .env را در Git commit نکنید.
- .gitignore را بررسی و تکمیل کنید.
- خروجی تست و logها را از نظر اطلاعات شخصی بررسی کنید.

bitwiti

- پیش از ارسال diff، وجود رازها را جست‌وجو کنید.
- اگر رازی افشا شد، آن را فقط حذف نکنید؛ در صورت نیاز rotate یا revoke کنید.

نمونهٔ .gitignore

```
gitignore
.env
.env.*
!.env.example
credentials/
secrets/
*.pem
*.key
__pycache__/
node_modules/
```

این نمونه باید با ساختار واقعی پروژه تطبیق داده شود.

فایل نمونهٔ امن

به‌جای نگهداری مقدار واقعی:

```
env
DATABASE_URL=postgresql://user:password@host/database
```

در فایل نمونه می‌توان نوشت:

```
env
DATABASE_URL=
API_KEY=
```

یا:

```
env
DATABASE_URL=replace-with-local-value
API_KEY=replace-with-local-value
```

مقدار واقعی باید خارج از ریپو و در محل امن مدیریت شود.

۶.۷. امنیت ورودی‌های کاربر

اگر Codex روی پروژه‌ای کار می‌کند که با دادهٔ کاربر، تصویر، صوت، پروندهٔ پزشکی یا اطلاعات هویتی سروکار دارد، حفاظت از داده‌ها اهمیت بیشتری پیدا می‌کند.

قواعد پیشنهادی:

- از دادهٔ واقعی در تست‌ها استفاده نکنید، مگر با مجوز و ناشناس‌سازی مناسب.
- نمونه‌های تست را مصنوعی یا ناشناس‌شده نگه دارید.
- شناسه‌های شخصی را در logها چاپ نکنید.
- فایل‌های بارگذاری‌شده را از نظر نوع و اندازه اعتبارسنجی کنید.
- دسترسی به داده را بر اساس کمترین سطح لازم تنظیم کنید.
- خروجی ابزارهای هوش مصنوعی را قبل از استفاده در سامانهٔ حساس بازبینی کنید.

در سامانه‌های پزشکی، فقط درست‌بودن کد کافی نیست؛ حریم خصوصی، قابلیت ردیابی، کنترل دسترسی و اعتبارسنجی بالینی نیز اهمیت دارند.

۶.۸. چک‌لیست امنیت پیش از شروع

.پوشهٔ درست پروژه انتخاب شده است [ ]

.مشخص است Git شاخهٔ فعلی [ ]

.بررسی شده است Git وضعیت اولیهٔ [ ]

.فایل‌های حساس شناسایی شده‌اند [ ]

.متناسب با کار است Codex سطح دسترسی [ ]

.عملیات نیازمند تأیید مشخص شده‌اند [ ]

.اجرای شبکه ضروری نیست یا کنترل شده است [ ]

.محدودهٔ فایل‌های مجاز و ممنوع مشخص است [ ]

.دستورهای خطرناک بدون بررسی اجرا نخواهند شد [ ]

۶.۹. چک‌لیست امنیت پس از پایان کار

.بررسی شده است git status [ ]

.از نظر رازها بررسی شده است git diff [ ]

.فایل جدید ناشناخته وجود ندارد [ ]

.فایل حساسی تغییر نکرده است [ ]

.یا تست اطلاعات خصوصی ندارد log خروجی [ ]

.انجام نشده است destructive عملیات [ ]

.تست‌ها اجرا و نتیجه ثبت شده است [ ]

.فقط پس از تأیید انسانی انجام می‌شود commit [ ]

۶.۱۰. خلاصهٔ فصل ششم

برای کار امن با Codex:

1. از کمترین سطح دسترسی لازم استفاده کنید.
2. قبل از اجرای فرمان‌های مهم، اثر آن‌ها را بررسی کنید.
3. تغییرات فایل و Git را مستقل از گزارش Codex کنترل کنید.
4. اطلاعات حساس را وارد گفت‌وگو یا diff نکنید.
5. عملیات حذف، بازنشانی و push اجباری را بدون بررسی انجام ندهید.
6. برای پروژه‌های دارای دادهٔ شخصی یا پزشکی، کنترل‌های اضافی اعمال کنید

فصل هفتم: Git و سه بررسی ضروری پیش از Commit

Git فقط ابزاری برای ذخیرهٔ نسخه‌های مختلف کد نیست؛ بلکه مهم‌ترین لایهٔ کنترل تغییرات هنگام کار با Codex است. هر تغییری که Codex ایجاد می‌کند باید از طریق وضعیت Git، diff و تست‌ها قابل مشاهده و ارزیابی باشد.

چرخهٔ پیشنهادی:

تغییر

↓

بررسیstatus

↓

بررسیdiff

↓

اجرای تست

↓

بازبینی امنیتی

↓

Commit

هرگز صرفاً به این دلیل که Codex اعلام کرده «کار تمام شد»، commit انجام ندهید.

۷.۱. وضعیت Git پیش از شروع

قبل از هر درخواست تغییردهنده، وضعیت پروژه را بررسی کنید:

```
bash
git status
```

برای خروجی کوتاه‌تر:

```
bash
git status --short
```

اگر تغییرات قبلی وجود دارد، آن‌ها را از تغییرات جدید جدا کنید. این موضوع مخصوصاً زمانی مهم است که پروژه را از قبل ویرایش کرده باشید.

نمونه:

M src/api/client.py

?? notes/debug.txt

پیش از ادامه، مشخص کنید:

- کدام تغییرات مربوط به کار قبلی هستند؟
- کدام فایل‌ها باید بدون تغییر باقی بمانند؟
- آیا تغییرات فعلی باید commit، stash یا جداگانه بررسی شوند؟
- آیا Codex ممکن است آن‌ها را با تغییر جدید مخلوط کند؟

در صورت وجود تغییرات مهم، به Codex بگویید:

تغییرات موجود در Git مربوط به کار قبلی هستند.

آن‌ها را تغییر نده و فقط روی فایل‌های مشخص‌شده در درخواست جدید کار کن.

۷.۲. بررسی اول: فایل‌های تغییرکرده

پس از اتمام کار، ابتدا فهرست فایل‌های تغییرکرده را ببینید:

```
bash
git status --short
```

از خود بپرسید:

- آیا فقط فایل‌های مورد انتظار تغییر کرده‌اند؟
- آیا فایل جدیدی بدون درخواست ایجاد شده است؟
- آیا فایل مهمی حذف یا جابه‌جا شده است؟
- آیا فایل‌های build، cache یا موقت ایجاد شده‌اند؟

- آیا تغییرات قبلی با تغییر جدید ترکیب شده‌اند؟

نمونهٔ مشکوک

فرض کنید درخواست شما فقط دربارهٔ اعتبارسنجی login بوده، اما خروجی چنین است:

M src/auth/login.py

M src/auth/service.py

M package-lock.json

M README.md

?? tmp/debug-output.log

در این شرایط، پیش از ادامه باید علت هر فایل مشخص شود. تغییر در package-lock.json، مستندات یا فایل log ممکن است منطقی باشد، اما نباید بدون بررسی پذیرفته شود.

پاک‌سازی فایل‌های موقت

فایل‌های موقت را ابتدا شناسایی کنید و سپس با احتیاط حذف کنید. از اجرای کورکورانهٔ این دستور خودداری کنید:

```
bash
git clean -fd
```

پیش از حذف می‌توانید فقط فهرست فایل‌های قابل حذف را ببینید:

```
bash
git clean -nd
```

گزینهٔ -n حالت آزمایشی است و عملیات حذف را انجام نمی‌دهد.

۷.۳. بررسی دوم: مشاهدهٔ diff

فهرست فایل‌ها کافی نیست؛ باید محتوای تغییر را نیز بخوانید:

```
bash
git diff
```

برای دیدن خلاصه:

```
bash
git diff --stat
```

برای مشاهدهٔ یک فایل خاص:

```
bash
git diff -- src/auth/login.py
```

هنگام بررسی diff به چه چیزهایی توجه کنیم؟

۱. تغییرات خارج از هدف

Codex بخش‌هایی را تغییر داده که در درخواست وجود نداشته‌اند؟

۲. تغییرات قالب‌بندی گسترده

گاهی یک formatter کل فایل را تغییر می‌دهد و تشخیص تغییر واقعی دشوار می‌شود. اگر قالب‌بندی کامل فایل ضروری نبوده، آن را جداگانه بررسی کنید.

۳. حذف ناخواسته

خطوط حذف‌شده را با دقت بخوانید؛ حذف یک import، شرط یا بررسی امنیتی ممکن است در ظاهر ساده به نظر برسد اما رفتار برنامه را تغییر دهد.

۴. تغییر پیام‌ها و قراردادها

تغییر متن خطا، نام فیلد JSON، کد HTTP یا امضای تابع ممکن است مصرف‌کنندگان دیگر را تحت‌تأثیر قرار دهد.

۵. اضافه‌شدن وابستگی

هر package جدید باید دلیل، منبع و ریسک مشخص داشته باشد.

۶. اطلاعات حساس

به دنبال این موارد بگردید:

API keys

passwords

tokens

private URLs

connection strings

personal data

برای جست‌وجوی اولیه، می‌توانید از عبارت‌های عمومی استفاده کنید؛ اما نتیجه را دستی بررسی کنید:

```bash
rg -n "password|secret|token|api[_-]?key" .
```

این جست‌وجو ممکن است false positive داشته باشد و جایگزین بررسی انسانی نیست.

۷.۴. بررسی سوم: تست و اعتبارسنجی

پس از بررسی diff، تست مناسب را اجرا کنید. ابتدا از دستور تعریف‌شده در پروژه استفاده کنید.

نمونه‌ها:

bash

pytest

bash

npm test

bash

flutter test

bash

go test ./...

علاوه بر تست، بسته به پروژه ممکن است این بررسی‌ها لازم باشند:

bash

ruff check .

bash

mypy src

bash

npm run lint

bash

npm run typecheck

تست موفق به‌تنهایی کافی نیست

سبزشدن تست‌ها تضمین می‌کند که:

- فایل درست تغییر کرده است.
- نیازمندی کاربر کامل اجرا شده است.
- رفتارهای مرزی پوشش داده شده‌اند.
- امنیت حفظ شده است.
- API قبلی نشکسته است.

بنابراین تست باید در کنار status، diff و بازبینی استفاده شود.

۷.۵. Staging مرحله‌ای

بهتر است همهٔ تغییرات را یک‌باره stage نکنید. فایل‌های مرتبط را مشخص انتخاب کنید:

bash
git add src/auth/login.py tests/test_login.py

سپس diff مربوط به stagedها را بررسی کنید:

bitwithit

```bash
git diff --cached
```

اگر اشتباهاً فایلی را stage کردید، می‌توانید آن را از staging خارج کنید، بدون اینکه تغییر محلی حذف شود:

```bash
git restore --staged path/to/file
```

تفاوت مهم

- git restore --staged فایل را از staging خارج می‌کند.
- git restore path/to/file ممکن است تغییرات محلی فایل را بازگرداند؛ بنابراین باید با احتیاط استفاده شود.
- git reset --hard می‌تواند تغییرات ثبت‌نشده را حذف کند و پرریسک است.

۷.۶. نوشتن پیام Commit

پیام commit باید نتیجهٔ تغییر را کوتاه و روشن بیان کند:

```bash
git commit -m "Fix login validation"
```

پیام ضعیف:

changes

پیام بهتر:

Fix invalid phone validation in signup form

در پروژه‌های تیمی، از قالب قراردادی پروژه پیروی کنی؛ برای مثال:

feat: add profile search

fix: handle expired access token

test: cover anonymous profile access

refactor: simplify image preprocessing

docs: update Codex workflow

Commit خوب چه ویژگی‌هایی دارد؟

- یک هدف مشخص دارد.
- تغییرات نامرتبط را مخلوط نمی‌کند.
- قابلیت بازبینی و بازگشت دارد.

- پیام آن برای آینده قابل فهم است.
- نتیجهٔ تغییر را بیان می‌کند، نه فقط فرایند را.

۷.۷. رفتار پیشنهادی Codex در Git

به‌طور پیش‌فرض، به Codex اجازه ندهید بدون تأیید شما عملیات زیر را انجام دهد:

- commit
- push
- merge
- rebase
- force push
- حذف شاخه
- reset
- clean
- حذف فایل‌های گسترده

نمونهٔ دستور امن:

کد و تست‌ها را آماده کن، اماcommit ، push، mergeو تغییر تاریخچهٔ Git را انجام نده.

در پایان، دستورهای پیشنهادی را فقط به‌صورت متن ارائه کن.

اگر از Codex خواستید commit انجام دهد، پیام commit، فایل‌های staged و نتیجهٔ تست را پیش از تأیید بررسی کنید.

۷.۸. بازگردانی تغییرات با احتیاط

اگر تغییر مناسبی ایجاد نشده است، ابتدا وضعیت را ثبت کنید:

```
bash
git status
```

سپس diff را ذخیره یا بررسی کنید. برای کنارگذاشتن تغییرات، روش مناسب به وضعیت فایل بستگی دارد.

فایل unstaged

```
bash
git restore path/to/file
```

این دستور تغییرات محلی فایل را کنار می‌گذارد. فقط زمانی استفاده کنید که مطمئن هستید تغییر لازم نیست.

فایل staged

ابتدا آن را از staging خارج کنید:

```
bash
git restore --staged path/to/file
```

سپس در صورت اطمینان، تغییر محلی را بازگردانید.

قانون ایمنی

هرگاه احتمال می‌دهید تغیر محلی مهم است، قبل از بازگردانی یکی از این کارها را انجام دهید:

- از فایل نسخهٔ پشتیبان بگیرید.
- patch تهیه کنید.
- commit موقت ایجاد کنید.
- تغییر را در یک شاخهٔ جدا نگه دارید.

۷.۹. چک‌لیست نهایی پیش از Commit

.مشخص است commit هدف [ ]

.شاخهٔ فعلی درست است [ ]

.تغییرات قبلی از تغییرات جدید جدا شده‌اند [ ]

.فقط فایل‌های مرتبط تغییر کرده‌اند [ ]

.کامل خوانده شده است diff [ ]

.فایل حساس یا راز افشاشده وجود ندارد [ ]

.تست‌های مرتبط موفق بوده‌اند [ ]

.لازم اجرا شده است type-check یا lint [ ]

.تغییرات مخرب یا غیرقابل‌بازگشت انجام نشده‌اند [ ]

.واضح است commit پیام [ ]

.جداگانه و آگاهانه انجام خواهد شد push [ ]

bitwitit



فصل هشتم: طراحی و مدیریت AGENTS.md

فایل AGENTS.md راهنمای محلی رفتار Codex در یک پروژه است. این فایل به‌جای تکرار همهٔ مستندات، باید تصمیم‌های اجرایی و قوانین مهم پروژه را در اختیار عامل قرار دهد.

هدف AGENTS.md این نیست که Codex را با متن طولانی محدود کند؛ هدف آن کاهش ابهام و استانداردکردن روش کار است.

۸.۱. چه چیزهایی در AGENTS.md قرار می‌گیرند؟

محتوای مناسب معمولاً شامل این موارد است:

- معرفی کوتاه پروژه
- ساختار مهم پوشه‌ها
- دستور نصب

- دستور اجرای برنامه
- دستور تست
- دستور lint و type-check
- قوانین معماری
- محدودیت‌های تغییر
- قواعد امنیتی
- سیاست commit و push
- روش گزارش نتیجه
- دستورهای ممنوع یا نیازمند تأیید

۸.۲. محل قرارگیری فایل‌ها

می‌توان در سطح ریشهٔ پروژه یک AGENTS.md داشت و برای زیرپروژه‌ها یا پوشه‌های تخصصی نیز فایل‌های جدا ایجاد کرد.

۸.۵. بایدها و نبایدها

بایدها

- دستورهای واقعی و قابل اجرا بنویسید.
- قوانین را کوتاه و اولویت‌بندی‌شده نگه دارید.
- مسیر فایل‌ها را دقیق ذکر کنید.
- تصمیم‌های معماری مهم را ثبت کنید.
- دستورهای تست را به‌روز نگه دارید.
- قوانین امنیتی را صریح بنویسید.
- از مثال‌های واقعی پروژه استفاده کنید.

نبایدها

- رازها، توکن‌ها یا رمزها را وارد فایل نکنید.
- محتوای README را بدون نیاز تکرار نکنید.
- قوانین متناقض ایجاد نکنید.
- متن بسیار طولانی و مبهم ننویسید.
- دستورهای قدیمی را باقی نگذارید.

bitwith.it

- از AGENTS.md برای پنهان‌کردن رفتار ناامن استفاده نکنید.
- بدون درک ساختار پروژه، قوانین معماری اختراع نکنید.

فصل نهم: مفاهیم پیشرفتهٔ Codex

Codex فقط یک محیط گفت‌وگویی برای تولید یا اصلاح کد نیست. در پروژه‌های بزرگ‌تر، می‌توان آن را با دستورالعمل‌های تخصصی، عامل‌های فرعی، ابزارهای متصل و گردش‌کارهای تکرارشونده ترکیب کرد.

چهار مفهوم مهم در این فصل عبارت‌اند از:

Skill

Subagent

Plugin

MCP

این مفاهیم شباهت‌هایی با یکدیگر دارند، اما نقش آن‌ها یکسان نیست. درک تفاوت میان آن‌ها باعث می‌شود برای هر مسئله، ابزار درست را انتخاب کنید.

۹.۱. Skill چیست؟

Skill یا مهارت، یک بستهٔ دستورالعمل، دانش تخصصی و الگوی اجرایی برای انجام یک نوع کار تکرارشونده است.

نمونهٔ Skillها:

- بازبینی امنیتی API
- تولید تست برای کد Python
- تحلیل تصاویر پزشکی
- بررسی فایل‌های Flutter
- تهیهٔ مستندات endpointها
- بازبینی migration پایگاه داده
- تبدیل یک الگوی کدنویسی به الگوی دیگر
- تولید گزارش استاندارد release

Skill الزاماً یک مدل جدید یا برنامهٔ جداگانه نیست؛ بلکه مجموعه‌ای از دانش و قواعد سازمان‌یافته است که Codex هنگام نیاز از آن استفاده می‌کند.

۹.۲. Progressive Disclosure در Skill

یکی از اصول مهم طراحی Skill، افشای تدریجی یا Progressive Disclosure است.

یعنی تمام جزئیات Skill از ابتدا وارد زمینهٔ مدل نشود؛ بلکه اطلاعات در چند سطح ارائه شود

چرا افشای تدریجی مهم است؟

اگر تمام مستندات پروژه، مثال‌ها و قوانین از ابتدا وارد زمینه شوند:

- زمینهٔ مکالمه سریع‌تر پر می‌شود.
- اطلاعات مهم در میان جزئیات گم می‌شود.
- زمان تحلیل افزایش می‌یابد.
- سهمیه و منابع بیشتری مصرف می‌شود.
- احتمال استفاده از دستورالعمل نامرتبط بیشتر می‌شود.

۹.۵. Subagent چیست؟

Subagent یا عامل فرعی، یک عامل تخصصی‌تر است که برای انجام بخشی از کار به‌صورت مستقل یا نیمه‌مستقل به کار گرفته می‌شود.

Subagent می‌تواند:

- مسئله‌ای را جداگانه تحلیل کند.
- یک بخش مشخص از پروژه را بررسی کند.
- راه‌حل جایگزین پیشنهاد دهد.
- تست‌های خاص را اجرا یا تحلیل کند.
- نتیجه را به عامل اصلی برگرداند.

۹.۶. چه زمانی از Subagent استفاده کنیم؟

Subagent برای کارهایی مناسب است که:

- قابل تفکیک هستند.
- مرز مشخص دارند.
- نیازمند تخصص متفاوت‌اند.
- می‌توانند مستقل بررسی شوند.
- خروجی آن‌ها برای تصمیم‌گیری عامل اصلی مفید است

موارد نامناسب

استفاده از Subagent برای هر کار کوچک می‌تواند نتیجهٔ معکوس داشته باشد:

- هماهنگی پیچیده‌تر می‌شود.
- هزینه و زمان افزایش می‌یابد.
- خروجی‌های متناقض تولید می‌شوند.
- مسئولیت تصمیم نهایی مبهم می‌شود.
- تغییرات هم‌زمان ممکن است با یکدیگر تداخل کنند.

۹. Plugin چیست؟

Plugin یا افزونه، بسته‌ای است که قابلیت‌های جدیدی به محیط یا گردش‌کار Codex اضافه می‌کند.

یک Plugin ممکن است شامل این موارد باشد:

- دستورهای جدید
- Skillها
- ابزارهای تخصصی
- قالب‌های آماده
- اتصال به سرویس‌ها
- فایل‌های پیکربندی
- اسکریپت‌های کمکی

۱۰. ملاحظات امنیتی Plugin

هر Plugin را مانند نصب یک نرم‌افزار خارجی در نظر بگیرید.

پیش از استفاده بررسی کنید:

- منبع Plugin معتبر است؟
- چه فایل‌هایی را می‌خواند؟
- آیا به شبکه دسترسی دارد؟
- چه فرمان‌هایی اجرا می‌کند؟

bitwit

- آیا وابستگی خارجی دارد؟
- آیا داده‌ها را به سرویس دیگری ارسال می‌کند؟
- آیا نسخه و تغییرات آن قابل ردیابی است؟
- آیا امکان غیرفعال‌کردن یا حذف آن وجود دارد؟

از نصب Plugin ناشناخته فقط به‌دلیل نام جذاب یا توصیهٔ غیرقابل‌بررسی خودداری کنید.

۹.۱۱. MCP چیست؟

MCP یا Model Context Protocol، یک پروتکل برای اتصال مدل به ابزارها و منابع خارجی است.

این اتصال می‌تواند دسترسی استانداردشده به منابعی مانند موارد زیر ایجاد کند:

- پایگاه داده
- فایل‌ها
- مستندات
- issue tracker
- سرویس‌های داخلی
- GitHub یا سامانه‌های مشابه
- ابزارهای تحلیل
- سرویس‌های سازمانی

۹.۱۳. خطرهای MCP

اتصال MCP می‌تواند خطرهایی ایجاد کند:

- افشای داده‌های داخلی
- دسترسی بیش از حد
- اجرای عملیات بدون تأیید
- خواندن اطلاعات شخصی
- ارسال داده به سرویس ثالث
- تغییر ناخواسته در سامانهٔ واقعی
- ترکیب داده‌های نامعتبر با تصمیم فنی

اصول استفادهٔ امن از MCP

---

1. فقط سرورهای شناخته‌شده را فعال کنید.

2. مجوزها را محدود نگه دارید.
3. دسترسی خواندن و نوشتن را جدا کنید.
4. عملیات تغییردهنده را نیازمند تأیید کنید.
5. داده‌های حساس را بدون نیاز در اختیار مدل نگذارید.
6. log و مسیر دسترسی را بررسی کنید.
7. محیط آزمایشی را از محیط واقعی جدا کنید.
8. اتصال‌های بلااستفاده را غیرفعال کنید.

۹.۱۴. انتخاب ابزار مناسب

از این جدول به‌عنوان راهنمای اولیه استفاده کنید:	
نیاز	**ابزار مناسب**
دستورالعمل تکرارشونده	**Skill**
تقسیم تحلیل یا کار	**Subagent**
بستهٔ قابلیت قابل نصب	**Plugin**
اتصال به سرویس خارجی	**MCP**
قوانین عمومی پروژه	**AGENTS.md**
اجرای تست یا فرمان محلی	**CLI**
بررسی تغییرات	**Git**

۹.۱۵. گردش‌کار پیشنهادی برای استفاده از قابلیت‌های پیشرفته

مرحلهٔ اول: تشخیص نوع مسئله

ابتدا بپرسید:

آیا این کار یک دستورالعمل تکرارشونده است؟

آیا نیاز به اتصال به ابزار خارجی دارد؟

آیا می‌توان آن را به بخش‌های مستقل تقسیم کرد؟

آیا به قابلیت نصب‌شونده نیاز دارد؟

مرحلهٔ دوم: انتخاب کم‌پیچیده‌ترین راه‌حل

ترتیب پیشنهادی:

دستور عادی

↓

قالب یا دستورالعمل کوتاه

↓

Skill

↓

Subagent

↓

MCPیاPlugin

از ابزار پیچیده‌تر فقط زمانی استفاده کنید که ابزار ساده‌تر کافی نباشد.

مرحلهٔ سوم: تعیین محدوده و مجوز

مشخص کنید:

- چه فایل‌ها یا منابعی قابل دسترسی‌اند؟
- چه عملیات‌هایی فقط خواندنی هستند؟
- چه عملیات‌هایی نیازمند تأییدند؟
- خروجی چگونه اعتبارسنجی می‌شود؟
- چه اطلاعاتی نباید به ابزار داده شود؟

مرحلهٔ چهارم: اعتبارسنجی خروجی

نتیجهٔ Skill، Subagent، Plugin یا MCP باید با شواهد پروژه بررسی شود:

فایل‌ها

تست‌ها

diff

log

مستندات

وضعیت سرویس

۹.۱۶. خطاهای رایج در قابلیت‌های پیشرفته

استفاده از ابزار پیچیده برای مسئلهٔ ساده

اگر یک درخواست کوتاه کافی است، ساختن Plugin یا Subagent اضافی فقط سربار ایجاد می‌کند.

اعتماد کامل به خروجی Subagent

Subagent ممکن است بخشی از زمینه را نبیند یا فرض اشتباه داشته باشد. خروجی آن باید بررسی شود.

دسترسی بیش از حد MCP

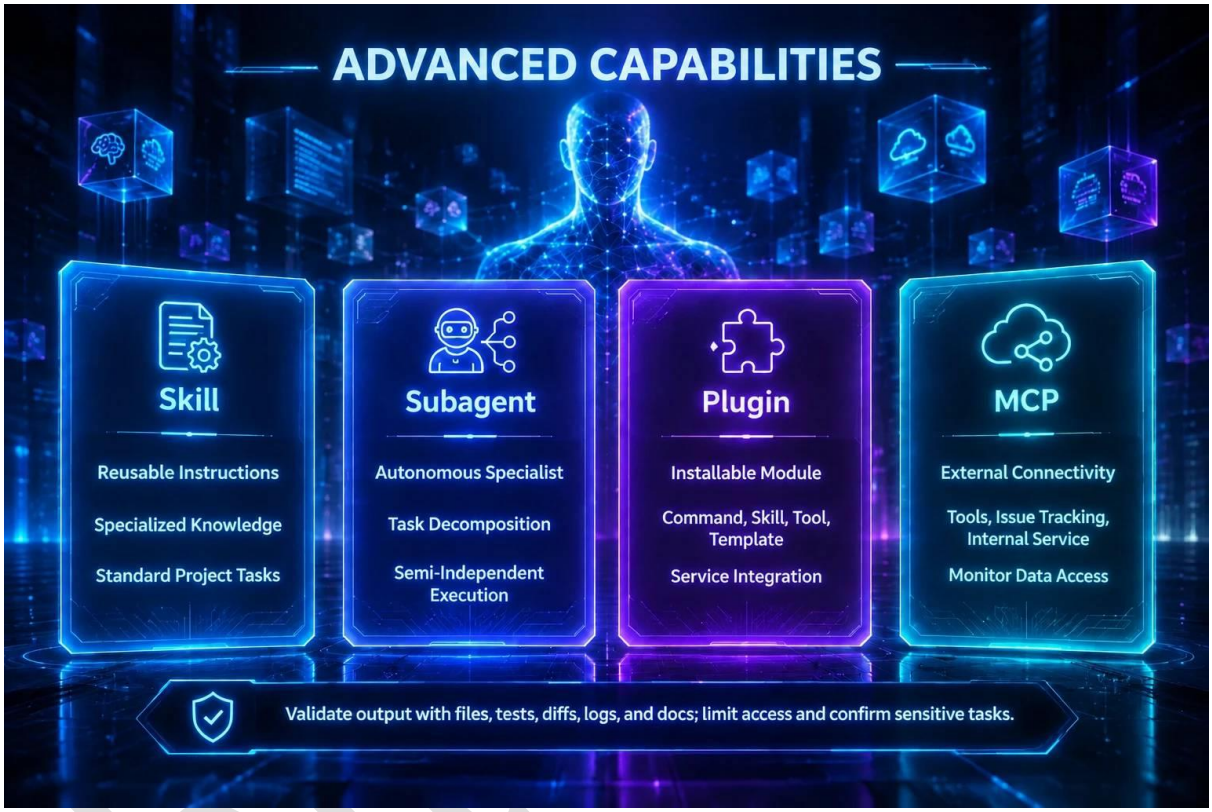
اتصال خواندن و نوشتن به محیط واقعی بدون محدودیت، ریسک جدی دارد.

Skill بیش‌ازحد طولانی

Skill باید اطلاعات موردنیاز را در زمان مناسب ارائه کند، نه اینکه تمام مستندات پروژه را یک‌جا بارگذاری کند.

قوانین متناقض

اگر AGENTS.md، Skill و دستور مستقیم کاربر با هم تعارض داشته باشند، اولویت‌ها و محدودهٔ هرکدام باید روشن شود



فصل دهم: انتخاب مدل و Reasoning Effort

انتخاب مدل باید بر اساس ماهیت کار انجام شود، نه صرفاً نام مدل یا بالاترین سطح توانایی آن.

مدل قوی‌تر همیشه انتخاب بهتری نیست. برای کارهای ساده، مدل سبک‌تر می‌تواند سریع‌تر، کم‌هزینه‌تر و کافی باشد. برای تغییرات معماری یا تحلیل ریسک، استفاده از مدل قدرتمندتر توجیه بیشتری دارد.

۱۰.۱. مدل‌ها به‌عنوان ابزار تخصصی

در این کتابچه سه نام نمونه برای مدل‌ها در نظر گرفته می‌شود:

Astra

Sol

Luna

این نام‌ها باید بر اساس قابلیت واقعی مدل‌ها در محیط شما تفسیر شوند؛ زیرا نام، سطح توانایی و دسترسی مدل‌ها ممکن است در نسخه‌های مختلف تغییر کند.

Astra

مناسب برای:

- تحلیل عمیق
- مسائل چندمرحله‌ای
- تغییرات معماری
- خطاهای پیچیده
- بررسی امنیتی
- تحلیل وابستگی میان چند بخش

Sol

مناسب برای:

- کارهای متعادل
- اصلاح‌های معمول
- تولید تست
- بازبینی کد
- مستندسازی فنی
- تحلیل‌های متوسط

Luna

مناسب برای:

- کارهای تکراری
- تغییرهای کوچک
- قالب‌بندی
- تولید متن ساده
- تبدیل‌های محدود
- کارهایی با ریسک پایین

این دسته‌بندی نمونه است؛ معیار نهایی باید رفتار واقعی مدل در پروژهٔ شما باشد.

bitwithit

۱۰.۲. سطوح Reasoning Effort

می‌توان میزان تلاش استدلالی را به شکل زیر تصور کرد:

Light

Medium

Heavy

Ultra

Light

برای:

- پاسخ کوتاه
- توضیح ساده
- تغییر یک خط یا تابع
- قالب‌بندی
- تولید متن کم‌ریسک

Medium

برای:

- تغییر چند تابع
- افزودن تست
- رفع خطای معمول
- بازبینی یک ماژول
- تحلیل وابستگی محدود

Heavy

برای:

- باگ چندفایلی
- تغییر API
- مهاجرت وابستگی
- بررسی امنیت
- تغییر معماری
- تحلیل رفتارهای مرزی

bitwithit

Ultra

برای:

- مسئلهٔ بسیار پیچیده
- تصمیم‌های معماری پرریسک
- تحلیل جامع سامانه
- تعارض‌های چندلایه
- کارهایی که نیازمند بررسی عمیق و چندمرحله‌ای هستند

از Ultra برای کارهای روزمره استفاده نکنید؛ زیرا ممکن است زمان و سهمیهٔ بیشتری مصرف کند بدون آنکه ارزش متناسب ایجاد کند.

۱۰.۴. روش سه‌مرحله‌ای انتخاب

مرحلهٔ اول: ریسک را بسنجید

آیا خطا فقط یک فایل را تحت‌تأثیر قرار می‌دهد یا می‌تواند داده، امنیت، API یا معماری را خراب کند؟

مرحلهٔ دوم: عدم‌قطعیت را بسنجید

آیا دقیقاً می‌دانید چه باید تغییر کند؟ اگر مسئله مبهم است، ابتدا برنامه‌ریزی و تحلیل بخواهید.

مرحلهٔ سوم: قابلیت بازگشت را بسنجید

آیا تغییر به‌راحتی قابل بازگشت است؟ تغییر migration یا schema معمولاً از تغییر متن مستندات کم‌قابل‌بازگشت‌تر است.

۱۰.۵. افزایش تدریجی عمق استدلال

بهتر است همیشه از بالاترین سطح شروع نکنید. روش پیشنهادی:

Light

↓ اگر کافی نبود

Medium

↓ اگر کافی نبود

Heavy

↓ فقط در صورت نیاز واقعی

Ultra

در هر مرحله بررسی کنید مشکل از کدام مورد است:

- مدل ضعیف است؟

- درخواست مبهم است؟
- فایل کافی در اختیار مدل نیست؟
- تست یا شواهد لازم وجود ندارد؟
- محدودیت‌ها مشخص نشده‌اند؟

گاهی اصلاح prompt بیشتر از افزایش Reasoning نتیجه می‌دهد.



فصل یازدهم: عیب‌یابی، کنترل کیفیت و تکمیل چرخهٔ کار

عیب‌یابی با Codex فقط به معنای اصلاح خطا نیست. باید مشخص شود خطا در کدام لایه قرار دارد:

دسته‌بندی خطاها

خطای درک مسئله

Codex هدف یا محدودیت را اشتباه فهمیده است.

نشانه‌ها:

- تغییر فایل‌های خارج از محدوده

- حل‌کردن مسئله‌ای متفاوت
- نادیده‌گرفتن معیار پذیرش
- فرض‌کردن رفتارهای اعلام‌نشده

خطای محیط

مشکل از مسیر، نسخهٔ ابزار، وابستگی یا مجوز است.

نشانه‌ها:

- فرمان در ترمینال دستی نیز شکست می‌خورد.
- فایل یا ابزار پیدا نمی‌شود.
- نسخهٔ Python، Node یا Flutter ناسازگار است.
- دسترسی نوشتن وجود ندارد.

خطای وابستگی

نشانه‌ها:

- package نصب نیست.
- lockfile ناسازگار است.
- نسخهٔ transitive تغییر کرده است.
- API یک کتابخانه تغییر کرده است.

راه‌حل:

- فایل manifest و lockfile را بررسی کنید.
- نسخه‌ها را ثبت کنید.
- پیش از نصب یا به‌روزرسانی، تغییرات مورد انتظار را مشخص کنید.
- وابستگی جدید را بدون دلیل اضافه نکنید.

خطای کد

نشانه‌ها:

- تست مشخصی شکست می‌خورد.
- traceback به مسیر مشخصی اشاره می‌کند.
- ورودی خاص رفتار نادرست ایجاد می‌کند.

راه‌حل:

1. خطا را بازتولید کنید.
2. کوچک‌ترین مثال ممکن را بسازید.

3. مسیر اجرای کد را دنبال کنید.
4. تستی برای رفتار نادرست اضافه کنید.
5. اصلاح محدود انجام دهید.
6. تست قبلی و تست جدید را اجرا کنید.

خطای محیط اجرا یا استقرار

نشانه‌ها:

- کد محلی موفق است اما محیط staging یا production شکست می‌خورد.
- متغیر محیطی موجود نیست.
- مجوز، شبکه یا سرویس خارجی متفاوت است.
- نسخهٔ runtime با محیط توسعه یکسان نیست.

چرخهٔ استاندارد رفع باگ

1. مشاهدهٔ خطا
2. بازتولید
3. محدودکردن دامنه
4. ساخت فرضیه
5. جمع‌آوری شواهد
6. نوشتن یا اصلاح تست
7. اعمال کوچک‌ترین اصلاح
8. اجرای تست
9. بررسی regression
10. ثبت نتیجه

خطاهای مربوط به مسیر و فایل

اگر Codex فایل اشتباه را تغییر می‌دهد:

ابتدا ریشهٔ Git و مسیر فایل هدف را مشخص کن.

فقط این فایل‌ها مجاز هستند:

- `[path]`
- `[path]`

bitwithit

هر فایل دیگری را بدون تأیید تغییر نده.

بررسی دستی:

```
bash
git rev-parse --show-toplevel
git status --short
find . -maxdepth 2 -type f
```

در پروژه‌های چندبخشی، مشخص کنید هدف backend، mobile، frontend یا زیرپروژهٔ دیگری است.

۱۲.۵. خطاهای مربوط به زمینهٔ گفت‌وگو

نشانه‌ها:

- Codex فرض‌های قدیمی را ادامه می‌دهد.
- پاسخ‌ها با وضعیت فعلی پروژه سازگار نیستند.
- فایل‌هایی را تغییر می‌دهد که دیگر مرتبط نیستند.
- خروجی‌ها تکراری یا پراکنده می‌شوند.

راه‌حل:

زمینهٔ فعلی را از روی فایل‌ها و Git بازسازی کن.

فرض‌های قبلی را معتبر تلقی نکن.

ابتدا این موارد را گزارش کن:

- شاخهٔ فعلی

- فایل‌های تغییرکرده

- هدف فعلی

- تست موجود

- محدودیت‌های فعلی

bitwitit